



Széchenyi István Katolikus Technikum és Gimnázium

Szoftverfejlesztő és -tesztelő projektfeladat



„CashTrack, az Ön költségvetés-kezelője”

Készítették:

Martocean Máté,

Pacza Dominik,

Porkoláb Martin

Ózd, 2025

Tartalom

BEVEZETÉS	4
FELHASZNÁLÓI DOKUMENTÁCIÓ	6
RENDSZERKÖVETELMÉNY	6
<i>Hardver</i>	<i>6</i>
<i>Szoftver</i>	<i>6</i>
WEBALKALMAZÁS ELINDÍTÁSA	7
WEBALKALMAZÁS HASZNÁLATA	8
<i>Általános információk a webalkalmazásról</i>	<i>8</i>
<i>Főoldal</i>	<i>8</i>
<i>Kapcsolat</i>	<i>9</i>
<i>Bejelentkezés</i>	<i>10</i>
<i>Regisztráció</i>	<i>11</i>
<i>Irányítópult</i>	<i>12</i>
FEJLESZTŐI DOKUMENTÁCIÓ	18
ALKALMAZOTT FEJLESZTŐI ÉS CSOPORTMUNKA ESZKÖZÖK	18
ADATBÁZIS	18
FRONTEND	18
BACKEND	18
ADATBÁZIS	19
MODELLEK LEÍRÁSA	19
TÁBLÁK, KAPCSOLATOK	21
E-K DIAGRAM	23
FRONTEND	24
ALKALMAZÁS FRONTEND RÉSZÉNEK RÉSZLETES LEÍRÁSA	24
<i>Főoldal</i>	<i>24</i>
<i>Kapcsolat</i>	<i>25</i>

<i>Rólunk</i>	26
<i>Bejelentkezés</i>	27
<i>Header</i>	28
<i>Register</i>	30
<i>Dashboard</i>	31
<i>Piechart</i>	32
<i>Modellek</i>	34
<i>Service-k</i>	34
RESPONZIVITÁS	37
<i>Kezdőlap</i>	37
<i>Bejelentkezés és Regisztráció</i>	37
<i>Rólunk</i>	38
<i>Irányítópult</i>	39
BACKEND	40
ALKALMAZÁS BACKEND RÉSZÉNEK RÉSZLETES LEÍRÁSA	40
API VÉGPONTOK.....	40
THUNDER CLIENT TELEPÍTÉSE	40
<i>”Felhasználó” lekérdezések</i>	40
<i>”Jövedelem” lekérdezések</i>	41
<i>”Jövedelem kategóriák” lekérdezések</i>	41
<i>”Kiadás” lekérdezések</i>	42
<i>”Kiadás kategóriák” lekérdezések</i>	42
<i>Hibák hibás lekérdezésekkor</i>	42
KONTROLLEREK	43
<i>FelhasznaloController</i>	43
<i>JovedelemController</i>	45
<i>JovedelemKategoriakController</i>	47

<i>KiadasController</i>	48
<i>KiadasKategoriakController</i>	50
TESZTELÉS	51
TOVÁBBFEJLESZTÉSI LEHETŐSÉGEK	54
FUNKCIONALITÁS BŐVÍTÉSE	54
FELHASZNÁLÓI ÉLMÉNY JAVÍTÁSA	54
BIZTONSÁG ÉS ADATVÉDELEM	55
BACKEND ÉS TECHNOLÓGIAI FEJLESZTÉSEK	55
IRODALOMJEGYZÉK	55

Bevezetés

A CashTrack költségkezelő egy webalkalmazás, amely lehetővé teszi a felhasználók számára, hogy egyszerűen kezeljék, nyomon követhessék és elemezzék pénzügyi kiadásaikat és bevételeiket. A webalkalmazás célja, hogy segítse a felhasználókat a pénzügyeik átláthatóságának növelésében, lehetővé téve számukra a költségvetésük részletes kezelését és esetleges rendszerezését.

Funkcionalitásban a felhasználók egyéni kategóriákba sorolhatják kiadásaikat, mint például étkezés, lakbér, szórakozás stb. A webalkalmazás külön lehetőséget biztosít a különböző típusú bevételek nyilvántartására is (pl. fizetés, bónusz), így a felhasználók pontos képet kaphatnak pénzügyi helyzetükről. A felhasználók könnyen rögzíthetik napi kiadásaikat, ha megadják azoknak típusát, összegét és dátumát. A rendszer figyelni és automatikusan frissíti a kategóriák költségkeretét, így a felhasználók könnyen nyomon követhetik, mennyit költöttek egy adott időszakban. Grafikus tekintetben is látszanak az adatok, megjeleníti a költségeket is és a kiadásokat is, grafikonok formájában. Ez segít a felhasználóknak gyorsan áttekinteni pénzügyeik állapotát.

Visszamenőlegesen is lesz lehetőség áttekinteni a havi költségeket, a felhasználó kiválaszt egy adott hónapot, majd áttekinthető lesz az összes kiadása és bevétele egyaránt.

Az oldal felhasználóbarát felülettel rendelkezik, így könnyedén segítheti az emberek minden napját.

Az oldal GitHub repositoryja: <https://github.com/hatarvedo/CashTrack-public>

Felhasználói dokumentáció

Rendszerkövetelmény

Hardver

	Minimum	Ajánlott
CPU	Intel Core i3 (4. generáció) vagy AMD Ryzen 3	Intel Core i5 (8. generáció) vagy AMD Ryzen 5
GPU	Integrált grafikus vezérlő (Intel HD Graphics 4000 vagy AMD Radeon Vega 3)	Intel Iris XE vagy AMD Radeon Vega 8
RAM	4 GB	8 GB vagy több
Tárhely	250 MB szabad hely	500 MB szabad hely
Internet	Széles sávú internetkapcsolat	Széles sávú internetkapcsolat
Operációs rendszer	Windows 10 macOS 10.14 Linux (Ubuntu 18.04+) vagy Android 8.0+ iOS 12+ (mobilon)	Windows 11 macOS 12 Linux (Ubuntu 22.04+) vagy Android 10+ iOS 15+
Böngésző	Google Chrome 88+ vagy Mozilla Firefox 85+	Legfrissebb verziójú Google Chrome Mozilla Firefox

Szoftver

- XAMPP ([Letöltés](#)) - 8.2.12 vagy újabb (aktuális verzió: 8.2.12 – 2025/02/06)
- Composer ([Letöltés](#)) – v.2.8.5 vagy újabb (aktuális verzió: v2.8.5 – 2025/02/06)

- PHP: 8.1 vagy újabb
- Git ([Letöltés](#)) – 2.48.1 vagy újabb (aktuális verzió: 2.48.1 – 2025/02/06)
- Node.js ([Letöltés](#)) - v18.19.1 vagy újabb (aktuális verzió: v23.10.0 - 2025/03/16)

Webalkalmazás elindítása

Backend beüzemelési folyamat

1. Telepítsük a XAMPP-ot, a Git-et, valamint a Composer-t a számítógépre.
2. Indítsuk el a XAMPP-ot, aztán a programon belül az Apache és MySQL szervereket.
3. Ha rendelkezünk mind a három programmal, nyissunk egy konzolos felületet. (pl. CMD, PowerShell). Írjuk be a következő parancsot:

```
composer global require laravel/installer
```

4. Migráljuk fel a táblákat a MySQL felületre a következő paranccsal:

```
php artisan migrate --seed
```

5. A sikeres migráció után futtassuk a szerveret a következő paranccsal:

```
php artisan serve
```

Frontend beüzemelési folyamat

1. Telepítsük a Node.js. (lásd: Követelmények)
2. Nyissuk meg a Visual Studio Code-t.
3. Nyissunk egy új terminált. (Ctrl+Shift+ö)
4. Írjuk be a telepítéshez szükséges parancsot.

```
npm install -g @angular/cli
```

5. Navigáljunk el a "CashTrack" mappába.

```
cd Frontend
```

```
cd CashTrack
```

6. Indítsuk el a szerveret.

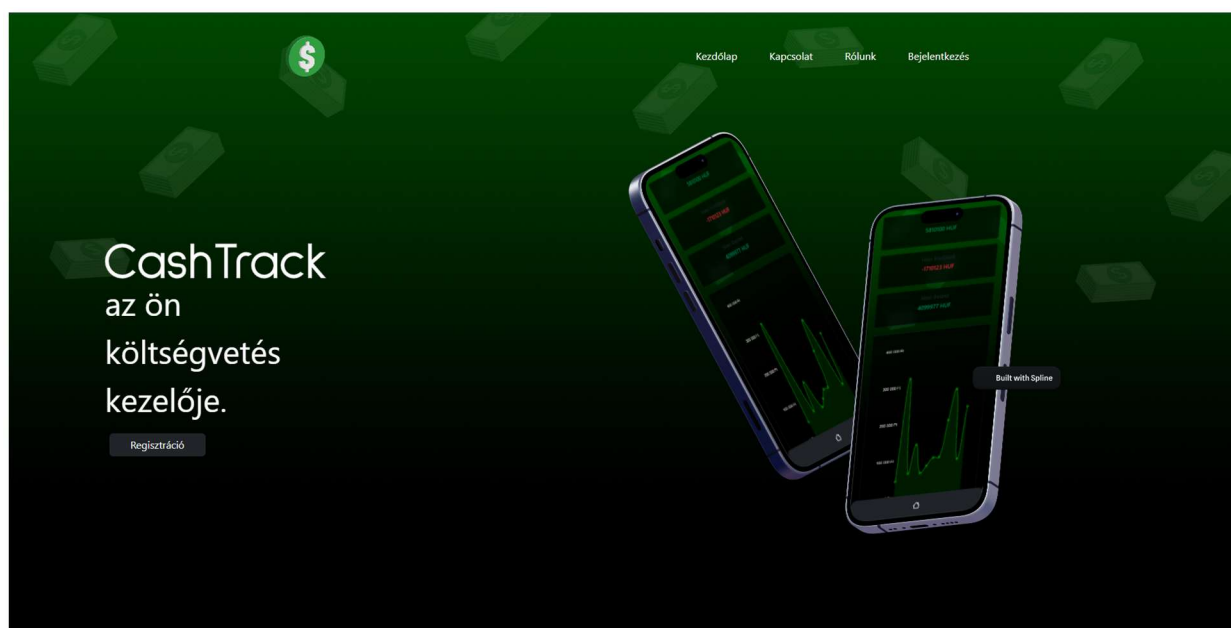
```
ng serve, vagy ng s
```

Webalkalmazás használata

Általános információk a webalkalmazásról

A **CashTrack** egy költségvetés-kezelő online webalkalmazás, amelyet Angular keretrendszerrel fejlesztettünk. Célja, hogy segítse a felhasználókat pénzügyeik rendszerezésében és átlátható nyomon követésében, különös tekintettel a kiadások és bevételek részletes kezelésére. A rendszer intuitív kezelőfelülettel, interaktív grafikonokkal és felhasználóbarát funkciókkal támogatja a pénzügyi tudatosság kialakítását.

Főoldal



1. kép (Főoldal)

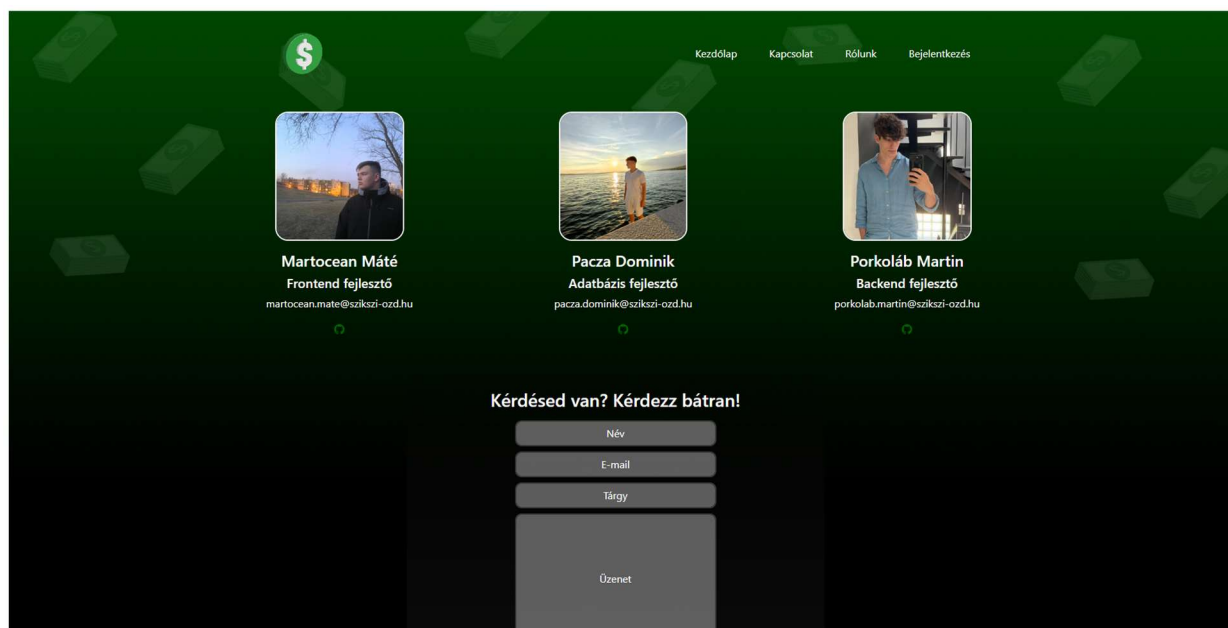
A képen látható a CashTrack főoldala, amely egy sötétzöldből feketébe irányuló színátmenetet kapott, valamint 3D-s pénzköteg ikonokat. Az elrendezése két részre oszlik: bal oldalon található a CashTrack felirat, egy „mottó” – „Az Ön költségvetés-kezelője”, valamint egy regisztráció gomb, jobb oldalon pedig egy illusztráció található, amelyet a [Spline](#) segítségével hoztunk létre, ezek lebegő effektust kaptak.

A felső navigációs menüben négy gombot találhatunk:

Kezdőlap, Kapcsolat, Rólunk, Bejelentkezés.

Az utóbbi gomb Irányítópulttá változik, ha a felhasználó be van jelentkezve.

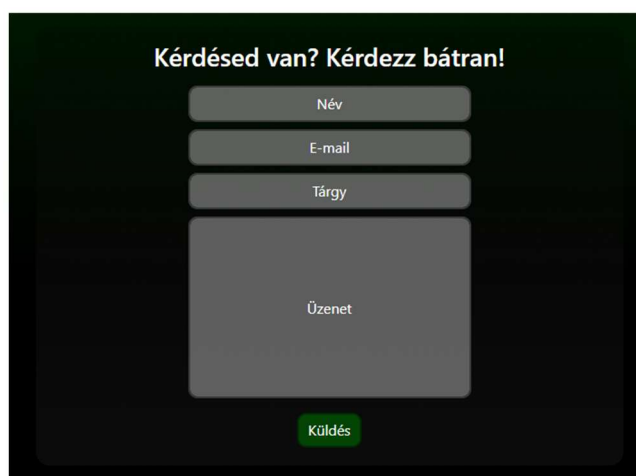
Kapcsolat



2. kép (Kapcsolat)

A képen látható a CashTrack kapcsolat oldala található, ez az oldal két részre oszlik: a fejlesztők, valamint egy űrlap.

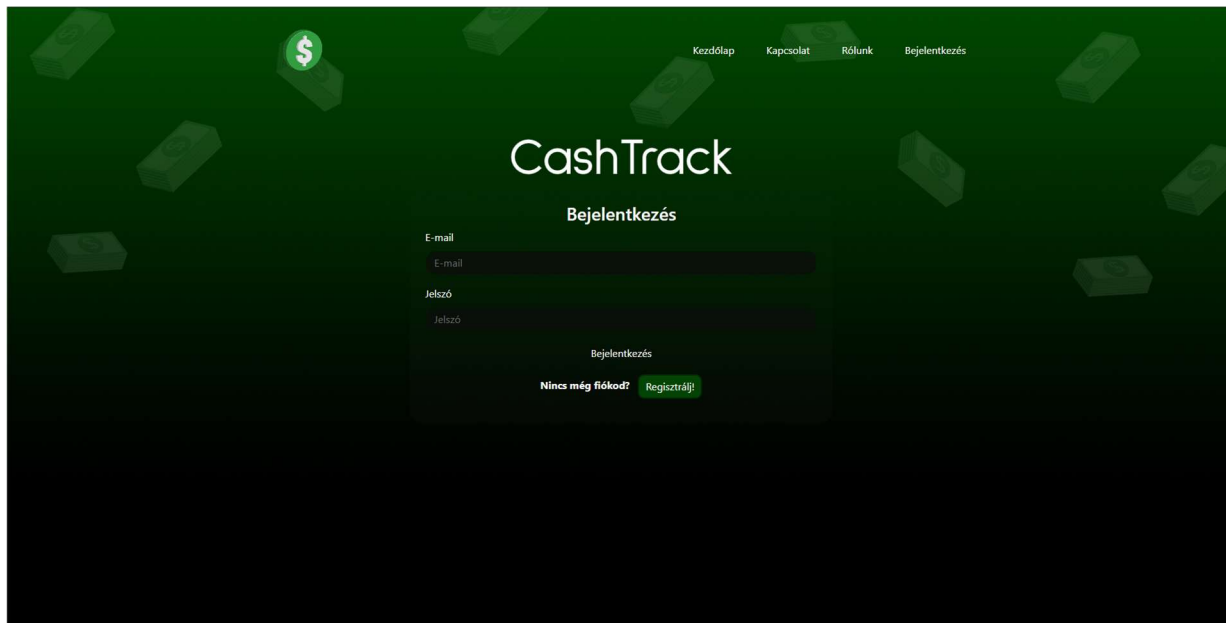
Az oldal felső részén a három fejlesztő személyes bemutatása látható, egymás mellett elhelyezve. A fejlesztők alatt megtalálhatóak a következők: név, szerepkör, e-mail cím, illetve egy GitHub ikon, amely a személyes GitHub oldalukra vezet.



3. kép (Kapcsolat)

A képen megtalálható egy egyszerű kapcsolatfelvételi űrlap, amelynek segítségével másodpercek alatt létesíthető kapcsolat a felhasználó és fejlesztők között. Az e-mail küldési szolgáltatást a [FormSubmit](#) biztosítja.

Bejelentkezés



4. kép (Bejelentkezés)

A bejelentkezési oldal központi része tartalmazza az alkalmazás nevét, az oldalcímet, valamint a bejelentkezéshez szükséges űrlapmezőket és gombokat.

A középre igazított „CashTrack” felirat nagy méretben jelenik meg, kiemelve az oldal célját. A „Bejelentkezés” alcím egyértelműen jelzi a felhasználó számára, hogy ezen az oldalon tud bejelentkezni fiókjába.

A következő űrlap mezők találhatóak az oldalon:

E-mail – kötelező mező, amelybe a felhasználó a regisztráció során megadott e-mail címét írja be.

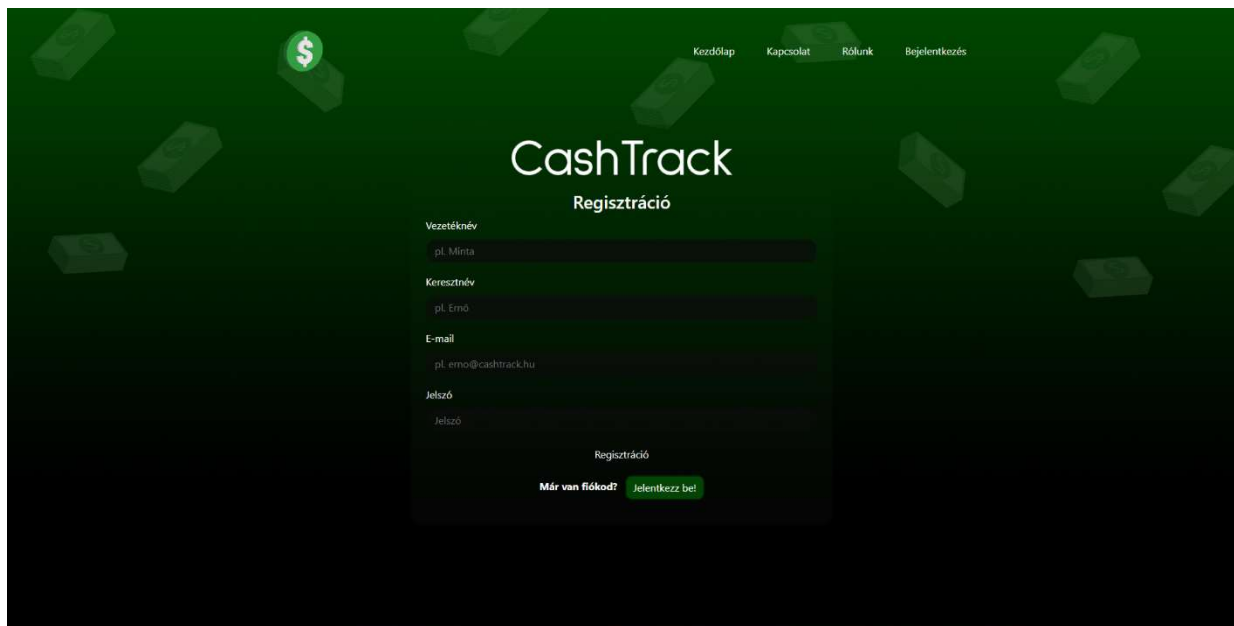
Jelszó – jelszó típusú mező, amely a hozzáférés védelmét szolgálja.

Bejelentkezés – az űrlap elküldését indítja el, és a megadott adatokkal kíséri meg az azonosítást.

Új felhasználók számára – Az űrlap alatt egy rövid felhívás („Nincs még fiókod?”) és egy „Regisztrálg!” gomb segíti a regisztrációs oldalra való navigációt.

A szekció célja az egyszerű, átlátható és gyors bejelentkezés biztosítása a felhasználók számára.

Regisztráció



4. kép (Regisztráció)

A regisztrációs oldal tartalmazza az alkalmazás nevét, az oldalcímet, valamint a regisztrációhoz szükséges űrlapmezőket és gombokat.

A középre igazított „CashTrack” felirat nagy méretben jelenik meg, kiemelve az oldal célját. A „Regisztráció” alcím egyértelműen jelzi a felhasználó számára, hogy ezen az oldalon tud regisztrálni egy új fiókot.

A következő űrlap mezők találhatóak az oldalon:

Vezetéknév – kötelező mező, amelybe a felhasználó a regisztráció során a vezetéknévét írja be.

Keresztnév – kötelező mező, amelybe a felhasználó a regisztráció során a keresztnév írja be

E-mail – kötelező mező, amelybe a felhasználó a regisztráció során megadott e-mail címét írja be.

Jelszó – jelszó típusú mező, amely a hozzáférés védelmét szolgálja.

Bejelentkezés – az űrlap elküldését indítja el, és a megadott adatokkal kíséri meg az azonosítást.

Régi felhasználók számára – Az űrlap alatt egy rövid felhívás („Már van fiókod?”) és egy „Jelentkezz be!” gomb segíti a bejelentkezési oldalra való navigációt.

Irányítópult



5. kép (Irányítópult)

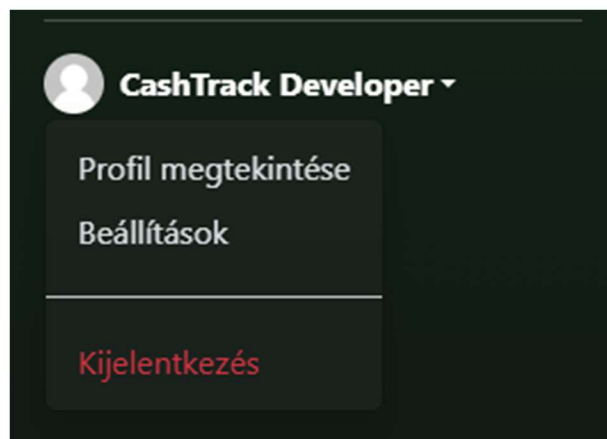
A webalkalmazás legfontosabb része, az Irányítópult, ahol nyomon tudjuk követni kiadásainkat, bevételeinket.

Az oldal bal oldalán található egy úgy nevezett „sidebar” (oldalsó navigációs menü) amelyet három részre bonthatunk.

Az első részen megtalálhatóak az alapértelmezett gombok (Kezdőlap, Rólunk, Kapcsolat).

A második részen megtalálhatóak a fontos interakciókhoz szükséges gombok (Bevétel hozzáadása, Kiadás hozzáadása).

A harmadik részen megtalálható egy lenyíló menü található ahol találhatóak menüpontok.



6. kép (Irányítópult)

A „Profil megtekintése” valamint a „Beállítások” gombok továbbfejlesztési lehetőségek, a „Kijelentkezés” gomb, kijelentkezteti a felhasználót és visszakerül a főoldalra.

Jövedelem 3919675 HUF	Kiadások -1847870 HUF	Összes 2071805 HUF
---------------------------------	---------------------------------	------------------------------

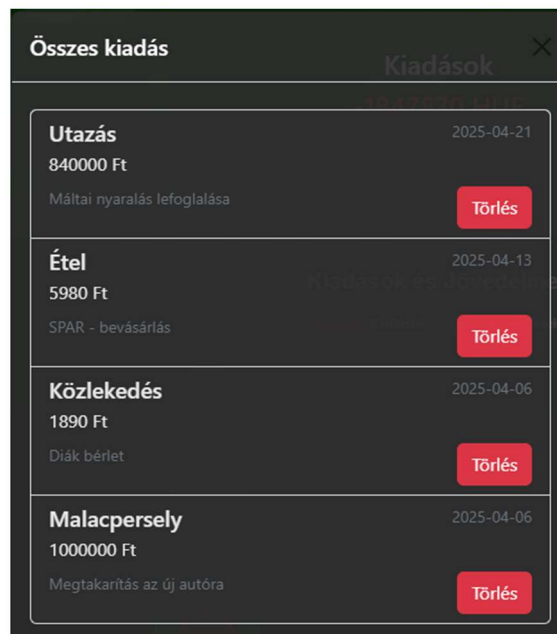
7. kép (Irányítópult)

Ezen a három kártyán megtalálhatóak az összesített jövedelmek, kiadások, valamint a kettő különbözete. Ezekkel a kártyákkal lehet interakcióba lépni, aminek a segítségével megnézhetjük a tranzakció történetünket.

Összes jövedelem		Kiadások
Hitelfelvétel		2025-04-18
3500000 Ft		
Törlés		
Fizetés		2025-04-10
413250 Ft		
Törlés		
Banki kamat		2025-04-02
6425 Ft		
Törlés		

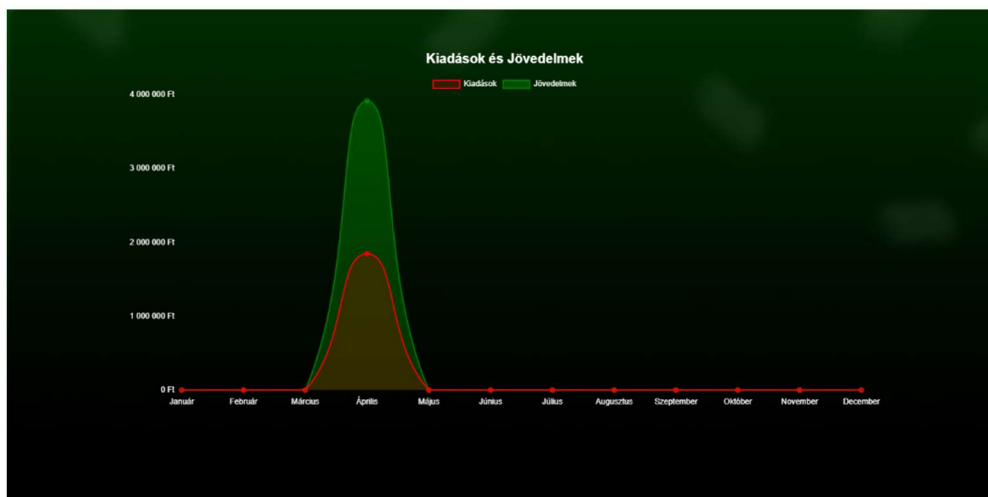
7. kép (Irányítópult)

A jövedelem kártyára kattintva előugrik a 7. képen látható ablak, ahol megnézhetjük egyesével az eddig történt bevételek típusát, összegét, dátumát, valamint egy törlés gombot.



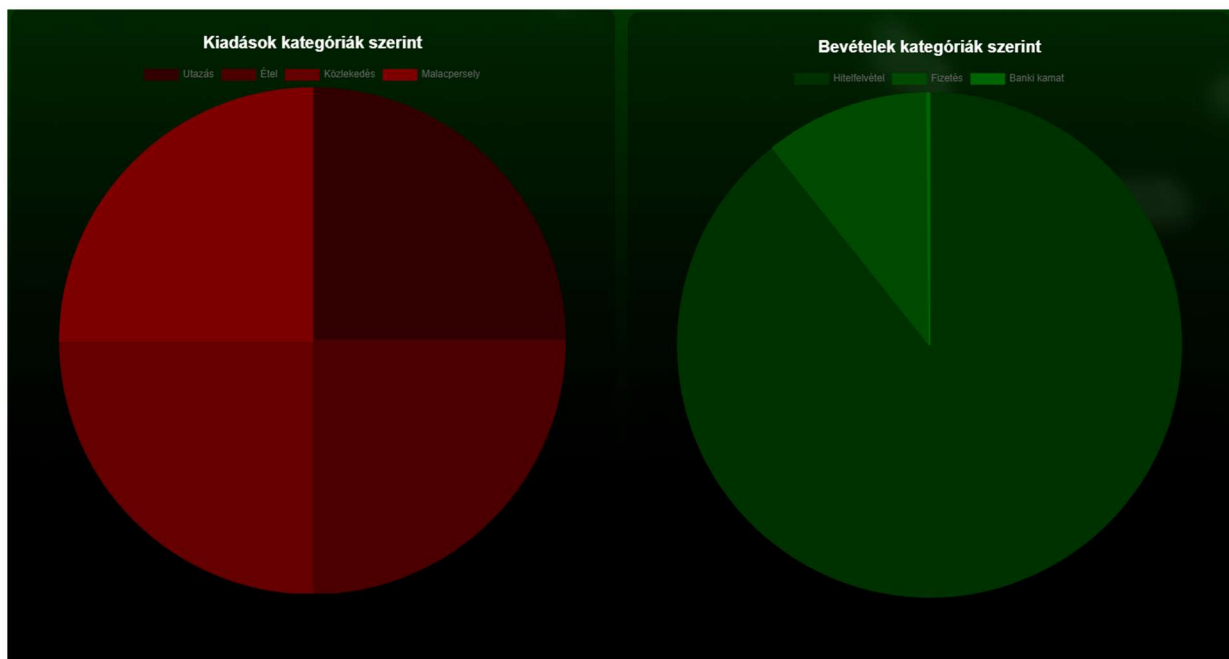
8. kép (Irányítópult)

A kiadás kártyára kattintva a 8. képen látható ablak jelenik meg, ahol megnézhetjük egyesével az eddig történt bevételek típusát, összegét, dátumát, egy törlés gombot, valamint egy kommentet, amit a felhasználó szabadon adhat meg. A komment megadása nem kötelező!



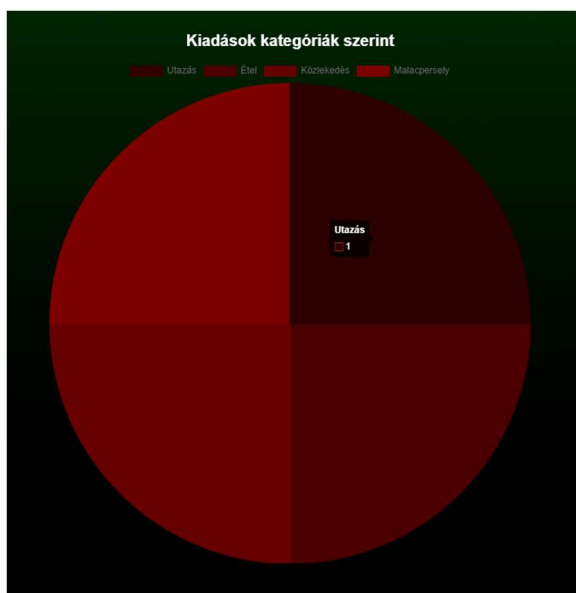
9. kép (Irányítópult)

A három kártya alatt megtalálható az első, legfontosabb grafikon, amin a kiadásainkat és jövedelmeinket nyomon követhetjük, hónapokra lebontva. A felső tengelyen találhatóak meg az összegek, az alsó tengelyen pedig a hónapok. A grafikonon található pöttyre, ha ráhúzzuk az egeret, akkor ki jön az adott hónapban történt kiadás vagy jövedelem összesítve.

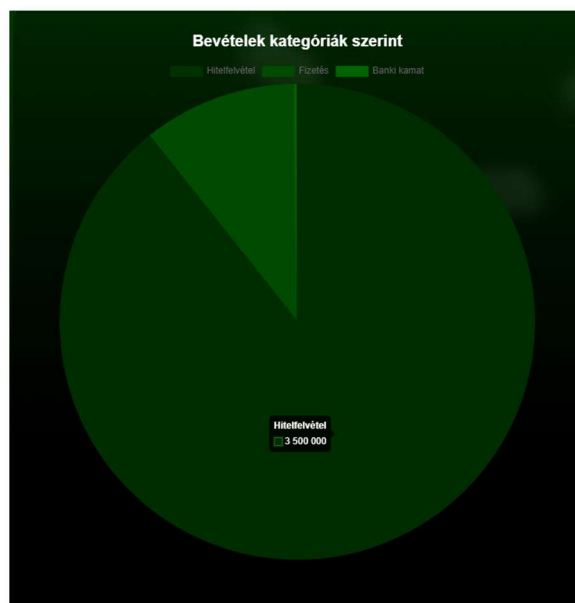


10. kép (Irányítópult)

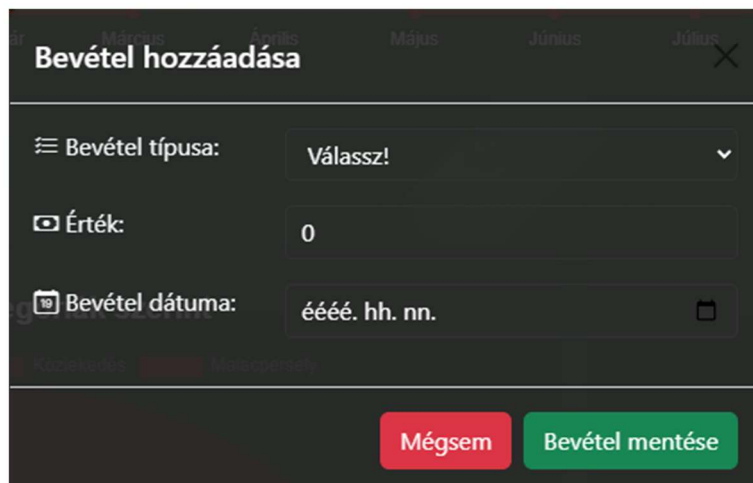
Az első grafikon alatt elhelyezkedik a kategóriák szerinti grafikon. A piros, „Kiadások kategóriák szerint” című grafikonon megtekinthetjük, hogy az adott kategóriában hányszor történt tranzakció. (11. kép) A zöld „Bevételek kategóriák szerint” című grafikonon megtekinthetjük, hogy az adott kategóriában mennyi pénz folyt be.



11. kép (Irányítópult)



12. kép (Irányítópult)



13. kép (Irányítópult)

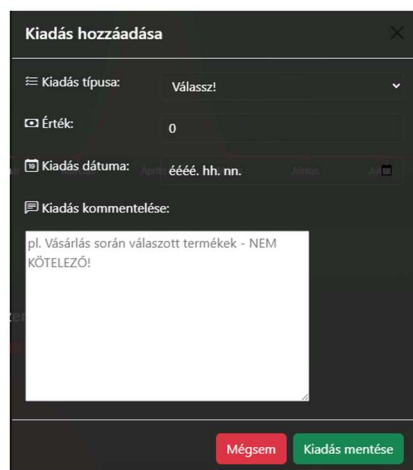
A 12. képen található a „Bevétel hozzáadása” ablak, aminek a segítségével a felhasználó egyszerűen vihet fel bevételt a rendszerbe.

A következő mezők találhatók az ablakon:

Bevétel típusa – A felhasználó előre választott típusokból választhat amelyek a következők: Fizetés, Banki kamat, Hitelfelvétel, Nyugdíj, Ösztöndíj, GYES, GYED, Egyéb

Érték – A felhasználó megadhatja a bevétel mennyiségét, amelyet a rendszer magyar Forintban tárol.

Bevétel dátuma – A felhasználó megadhatja a bevétel érkezési dátumát. Megadható előre, valamint utólagosan is a dátum.



14. kép (Irányítópult)

A 13. képen található a „Kiadás hozzáadása” ablak, aminek a segítségével a felhasználó egyszerűen vihet fel kiadást a

rendszerbe.

A következő mezők találhatóak meg a felületen:

Kiadás típusa – A felhasználó előre választott típusokból választhat amelyek a következők: Étel, Ház, Közlekedés, Telefonszolgáltató, Egészségügy, Ruházat, Higiénia, Gyerekek, Szórakozás, Utazás, Edzés, Megtakarítás, Malacpersely, Hiteltörlesztés, Támogatás, Egyéb

Érték – A felhasználó megadhatja a bevétel mennyiségét, amelyet a rendszer magyar Forintban tárol.

Kiadás dátuma – A felhasználó megadhatja a kiadás dátumát, amely megadható előre, valamint utólagosan is.

Fejlesztői dokumentáció

Alkalmazott fejlesztői és csoportmunka eszközök

A projektmunka során a **GitHub Desktop**-ot használtuk a kódok megosztására egymás között. Segítségével könnyedén kezelhettük a változtatásokat és nyomon követhettük a fejlesztés előrehaladását. A **Discord**-ot a csapaton belüli kommunikációra alkalmaztuk, hogy egyeztessünk a feladatokról. Az azonnali üzenetváltások révén hatékonyan tudtunk együttműködni. Ennek köszönhetően gördülékenyen haladtunk a projekt megvalósításával.

Adatbázis

A projektfeladat során az **adatbázis-kezeléshez** a **MySQL**-t használtuk, amely lehetővé tette az adatok hatékony tárolását és kezelését. A fejlesztői környezetként a **Visual Studio Code**-ot alkalmaztuk, amely segítette a kódírást és az adatbázis műveletek végrehajtását. A MySQL biztosította a megfelelő relációs adatbázis-struktúrát, míg a Visual Studio Code különböző bővítményekkel támogatta a fejlesztési folyamatot.

Frontend

A projektfeladat során a **frontend** fejlesztéséhez **HTML-t** használtunk az oldal szerkezetének kialakítására. A **CSS** segítségével biztosítottuk a megfelelő stílust és reszponzív megjelenést. Az **Angular** keretrendszert alkalmaztuk a dinamikus működés és az interaktív elemek megvalósítására. Az Angular komponensalapú felépítése lehetővé tette a kód könnyebb karbantarthatóságát és újrafelhasználhatóságát. A fejlesztés során az Angular biztosította az adatok hatékony kezelését és az oldal gyors frissítését. Ezek az eszközök együttműködve egy modern, jól strukturált és felhasználóbarát weboldalt eredményeztek.

Backend

A rendszer **backend** oldala **Laravel** keretrendszerrel készült, amely egy PHP-alapú, jól strukturált és biztonságos keretrendszer. A dokumentáció bemutatja a rendszer működését, API végpontjait, adatbázis-felépítését, valamint a telepítés lépéseit.

Adatbázis

A CashTrack adatbázisa a felhasználók adatait tárolja el egy költségvetési kalkulátorban. Az adatbázis eltárolja a felhasználók bevételeit, kiadásait dátum szerint. Ezt bárki használhatja és regisztrálhat az oldalra.

Modellek leírása

'felhasznalok' modell leírása

A 'felhasznaloID' az **elsődleges kulcs** a táblában, ez egy **integer** típus és ez a felhasználók egyedi azonosítója. Ez a mező egy **AUTO_INCREMENT** függvénnnyel rendelkezik, amely azt szolgálja, hogy minden egyes regisztrált fiók azonosítója eggyel növekszik.

A 'vezeteknev' **string** típus, ez a felhasználó vezetéknévét menti el a regisztrációkor.

A 'keresztnev' **string** típus, ez a felhasználó keresztnévét menti el a regisztrációkor. Az email a felhasználók bejelentkezéséhez azonosítást szolgál.

A 'jelszo' a felhasználók belépéséhez szükséges.

A 'profilkepUrl' a felhasználó profilképének megjelenítéséhez szükséges.

'jovedelemkategoriak' modell leírása

A 'kategoriaID' az **elsődleges kulcs** a táblában, egy **integer** típusú mező, amely a kategóriák egyedi azonosítója. Ez a mező egy **AUTO_INCREMENT** függvénnnyel rendelkezik, amely azt szolgálja, hogy minden egyes regisztrált fiók azonosítója eggyel növekszik.

A 'jovedelemKategoria' egy **string** típusú mező, amely a jövedelem pontos kategóriáját menti el. (pl. fizetés, GYES, GYED, stb.).

'jovedelmek' modell leírása

A 'jovedelemID' az **elsődleges kulcs** a táblában, egy **integer** típusú mező, amely a kategóriák egyedi azonosítója. Ez a mező egy **AUTO_INCREMENT** függvénnnyel rendelkezik, amely azt szolgálja, hogy minden egyes regisztrált fiók azonosítója eggyel növekszik.

A 'felhasznaloID' egy **idegen kulcs** a táblában, amely egy **integer** típusú mező, amely a felhasznalok táblában keresi meg az adott felhasználót, és ahhoz társítja az adott jövedelmet.

A 'bevetelMennyiseg' egy **integer** típusú mező, amellyel a felhasználó aktuális bevételét lehet felvinni az adatbázisba.

A 'bevetelDatum' egy **date** típusú mező, amellyel a bevétel mozgásának dátumát lehet naplózni. Ez a mező kap egy `current_timestamp()` tulajdonságot, amely a MySQL-ben a jelenlegi dátumot és időt adja vissza a rendszer órája alapján.

A 'kategoriaID' egy **idegen kulcs** a táblában, amely egy **integer** típusú mező, amely a jovedelemkategoriak táblában keresi meg az adott felhasználót, és társítja az adott kategóriát az adott bevételhez.

'kiadaskategoriak' modell leírása

A 'kategoriaID' az **elsődleges kulcs** a táblában, egy **integer** típusú mező, amely a kategóriák egyedi azonosítója. Ez a mező egy `AUTO_INCREMENT` függvénnnyel rendelkezik, amely azt szolgálja, hogy minden egyes regisztrált fiók azonosítója eggyel növekszik.

A 'kiadasKategoria' egy **string** típusú mező, amely a kiadás pontos kategóriáját menti el. (pl. számlabefizetés, vásárlások, előfizetések, stb.).

'kiadasok' modell leírása

A 'kiadasID' az **elsődleges kulcs** a táblában, egy **integer** típusú mező, amely a kategóriák egyedi azonosítója. Ez a mező egy `AUTO_INCREMENT` függvénnnyel rendelkezik, amely azt szolgálja, hogy minden egyes regisztrált fiók azonosítója eggyel növekszik.

A 'felhasznaloID' egy **idegen kulcs** a táblában, amely egy **integer** típusú mező, amely a felhasznalok táblában keresi meg az adott felhasználót, és ahhoz társítja az adott kiadást.

A 'kiadasHUF' egy **integer** típusú mező, amely a kiadás mennyiségét tárolja Forintban, ennek a segítségével akár amerikai dollárba, vagy euróba is lehet váltani, egy könnyű művelet segítségével.

A 'kiadasDatum' egy **date** típusú mező, amellyel a kiadás mozgásának dátumát lehet naplózni. Ez a mező kap egy `current_timestamp()` tulajdonságot, amely a MySQL-ben a jelenlegi dátumot és időt adja vissza a rendszer órája alapján.

A 'kategoriaID' egy **idegen kulcs** a táblában, amely egy **integer** típusú mező, amely a kiadaskategoriak táblában keresi meg az adott felhasználót, és társítja az adott kategóriát az adott kiadáshoz.

A 'kiadasKomment' egy **string** típusú mező, amellyel egy **opcionális** megjegyzést adhat a felhasználó a kiadás rögzítésekor.

Táblák, kapcsolatok

'felhasznalok' tábla felépítése:

Oszlop neve	Típus	Rövid leírás
felhasznaloID	id	Egyedi azonosító.
vezeteknev	string	Felhasználó vezetékneme.
keresztnev	string	Felhasználó keresztneme.
email	string	Felhasználó e-mail címe.
jelszo	string	Felhasználó jelszava.
profilkepUrl	string	Felhasználó egyedi profilképe.

'kiadas_kategoriak' tábla felépítése:

Oszlop neve	Típus	Rövid leírás
kategoriaID	id	Kategória azonosító.
kiadasKategoria	string	Kiadás kategória megnevezése.

'kiadas' tábla felépítése:

Oszlop neve	Típus	Rövid leírás
kiadasID	id	Kiadás azonosítója.
felhasznaloID	foreignId	Felhasználóhoz való hozzárendelés.
kiadasHUF	integer	Kiadás mennyisége.

kiadasDatum	date	Kiadás dátuma.
kiadasKategoria	foreignId	Kategóriához való hozzárendelés.
kiadasKomment	string	Kiadáshoz hozzáfűződő hozzászólás, ami nem kötelező.

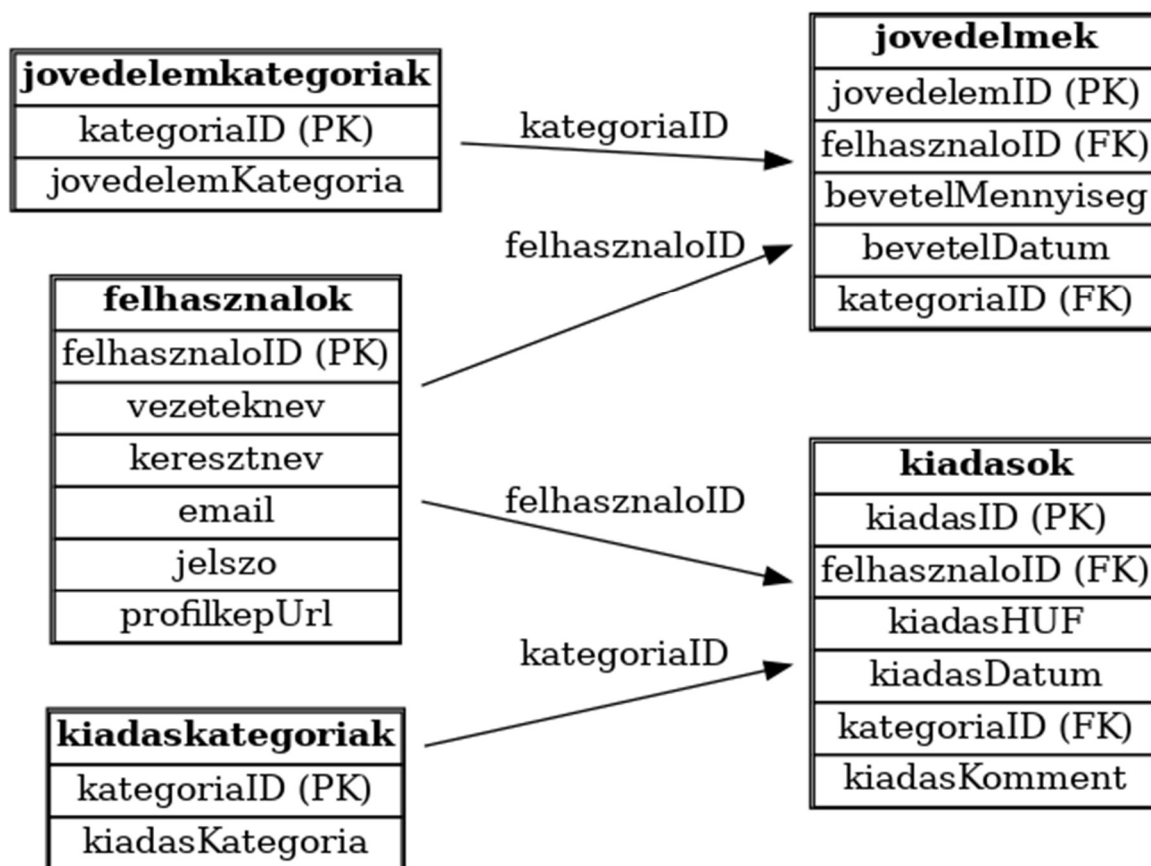
'jovedelem_kategoriak' tábla felépítése:

Oszlop neve	Típus	Rövid leírás
kategoriaID	id	Kategória azonosító.
jovedelemKategoria	string	Jövedelem kategória megnevezése.

'jovedelmek' tábla felépítése:

Oszlop neve	Típus	Rövid leírás
jovedelemID	id	Jövedelem azonosítója.
felhasznaloID	foreignId	Felhasználóhoz való hozzárendelés.
bevetelHUF	integer	Jövedelem mennyisége.
bevetelDatum	date	Jövedelem dátuma.
jovedelemKategoria	foreignId	Jövedelemhez való hozzárendelés.

E-K diagram



15. kép (E-K modell)

Frontend

Alkalmazás frontend részének részletes leírása

Dokumentációnk ezen része a CashTrack webalkalmazás fejlesztői oldalát részletesen mutatja be.

Főoldal

A főoldal felső részén található az úgynevezett navigációs sáv, ami egy külön Angular komponens, a **Header**. Ez a komponens minden komponensben megtalálható.

```
1 <div class="container">
2   <div class="header mt-4 mb-5" id="header">
3     <div class="logo-container col-5 col-lg-6 col-md-4 col-sm-5 inline">
4       <div data-aos="fade-down" data-aos-duration="1500">
5         <a routerLink="/" *ngIf="!loggedIn(); else dashPicture;">
6       </div>
7       <ng-template #dashPicture></ng-template>
8     </div>
9
10    <div class="nav-container">
11      <div class="desktop-menu">
12        <div data-aos="fade-down" data-aos-duration="2000">
13          <a routerLink="/home">Kezdőlap</a>
14        </div>
15        <div data-aos="fade-down" data-aos-duration="2000">
16          <a routerLink="/kapcsolat">Kapcsolat</a>
17        </div>
18        <div data-aos="fade-down" data-aos-duration="2500">
19          <a routerLink="/rolunk">Rólunk</a>
20        </div>
21        <div data-aos="fade-down" data-aos-duration="3000">
22          <a routerLink="/login" *ngIf="!loggedIn(); else dash;">Bejelentkezés</a>
23          <ng-template #dash data-aos="fade-down" data-aos-duration="3000">
24            <a routerLink="/dashboard">Irányítópult</a>
25          </ng-template>
26        </div>
27      </div>
28
29      <button class="hamburger-menu" (click)="toggleMenu()">
30        <i class="bi bi-list"></i>
31      </button>
32
33      <div class="mobile-menu" [class.active]="isMenuOpen">
34        <a routerLink="/">Főoldal</a>
35        <a routerLink="/kapcsolat">Kapcsolat</a>
36        <a routerLink="/rolunk">Rólunk</a>
37        <a routerLink="/login" *ngIf="!loggedIn(); else mobileDash;">Bejelentkezés</a>
38        <ng-template #mobileDash>
39          <a routerLink="/dashboard">Irányítópult</a>
40        </ng-template>
41      </div>
42    </div>
43  </div>
44 </div>
```

16. kép (Főoldal - Frontend)

Megtalálható benne a logónk, ami eltűnik, ha a felhasználó bejelentkezett, ezt egy ngIf használatával vizsgáljuk.

A navigációs sáv részeit az „AOS” (Animate On Scroll) használatával jeleníti meg az oldal, egy belebegés animációval. Ezeket sorban meghívjuk, 1500, 2000, 2500-as milliszekundumos késleltetéssel.

```

1 <app-header></app-header>
2 <div data-aos="fade-down" data-aos-duration="2500" class="contentbox">
3   <div class="doboz">
4     <div>
5       <p data-aos="fade-down" data-aos-duration="3000" class="szoveg">
6         <svg width="345" height="55" viewBox="0 0 345 55" fill="none" xmlns="http://www.w3.org/2000/svg"> -
7         </svg> az ön költségvetés <br> kezelője.</p>
8       <div data-aos="fade-down" data-aos-duration="1500" class="registerDiv">
9         <button data-aos="fade-down" data-aos-duration="1500" class="btn registerButton m-3 question" routerLink="/register">Regisztráció</button>
10      </div>
11    </div>
12  <div data-aos="fade-down" data-aos-duration="3000" class="spline-container">
13    <script type="module" src="https://unpkg.com/@splinetool/viewer@1.9.82/build/spline-viewer.js"></script>
14    <spline-viewer class="phone-3d" url="https://prod.spline.design/cUEVKIFayrPMdyQC/scene.splinecode"></spline-viewer>
15  </div>
16 </div>
17 </div>
18 </div>

```

17. kép (Főoldal - Frontend)

A főoldalon, azaz a **Home** komponensen a „CashTrack” felirat egy svg formátumú kód, amelyet CSS-ben animáltunk meg. Ezalatt a felirat alatt található egy regisztrációs gomb, amely átirányít minket a „**Register**” komponensre.

```

svg path {
  fill: transparent;
  stroke: #fff;
  stroke-width: 0.3;
  stroke-dasharray: 450;
  stroke-dashoffset: 450;
  animation: textAnimation 3s ease-in-out 1 forwards;
}

@keyframes textAnimation {
  0% {
    stroke-dashoffset: 450;
  }
  80% {
    fill: transparent;
  }
  100% {
    fill: #fff;
    stroke-dashoffset: 0;
  }
}

```

18. kép (Főoldal - Frontend)

A komponens másik fő eleme a 3D-s telefon „mockup”, amelyet a Spline nevű weboldal segítségével használhattunk. A modellt csak kiexportáltuk az általuk megadott kódokkal, és elhelyeztük a weboldalon CSS segítségével.

Kapcsolat

Ebben a komponensben lehet a fejlesztő csapattal felvenni a kapcsolatot, illetve található róluk némi információ is: nevük, kép róluk, github linkek a profilukhoz illetve az emailcímük.

```

1 <app-header>=</app-header>
2 <div class="container mt-5" style="color: white;">
3   <div data-aos="fade-down" data-aos-duration="2500" class="row">
4     <div class="col-4 textCenter">
5       
6       <br>
7       <h4 class="mt-3">Martocan Máté</h4>
8       <h5>Frontend fejlesztő</h5>
9       <p>martocan.mate@szikszizsiki-ozd.hu</p>
10      <p class="textCenter">
11        <a href="https://github.com/hatarvedo" target="_blank"><i class="bi bi-github"></i></a>
12      </p>
13    </div>
14    <div class="col-4 textCenter">
15      
16      <br>
17      <h4 class="mt-3">Pacsa Dominik</h4>
18      <h5>Adatbázis fejlesztő</h5>
19      <p>pacza.dominik@szikszizsiki-ozd.hu</p>
20      <p class="textCenter">
21        <a href="https://github.com/dominikpacza" target="_blank"><i class="bi bi-github"></i></a>
22      </p>
23    </div>
24    <div class="col-4 textCenter">
25      
26      <br>
27      <h4 class="mt-3">Porkolab Martin</h4>
28      <h5>Backend fejlesztő</h5>
29      <p>porkolab.martin@szikszizsiki-ozd.hu</p>
30      <p class="textCenter">
31        <a href="https://github.com/trashieeee" target="_blank"><i class="bi bi-github"></i></a>
32      </p>
33    </div>
34  </div>
35

```

19. kép (Kapcsolat - Frontend)

```

<div class="row justify-content-center mt-5">
  <div class="reportDiv col-12 col-md-8 col-lg-6 container pt-3 pb-3 textCenter mb-4">
    <h3>Kérdésed van? Kérdezz bátran!</h3>
    <form class="report" action="https://formsubmit.co/martocan.mate@szikszizsiki-ozd.hu" method="POST">
      <div class="d-flex justify-content-center">
        <input type="text" name="name" id="nev" placeholder="Név" class="form-control mt-2 w-50" required>
      </div>
      <div class="d-flex justify-content-center">
        <input type="text" name="email" id="email" placeholder="E-mail" class="form-control mt-2 w-50" required>
      </div>
      <div class="d-flex justify-content-center">
        <input type="text" name="Subject" id="targy" placeholder="Tárgy" class="form-control mt-2 w-50" required>
      </div>
      <div class="d-flex justify-content-center">
        <input type="text" name="message" id="uzenet" placeholder="Üzenet" class="form-control mt-2 w-50" required>
      </div>
      <input type="hidden" name="_captcha" value="false">
      <input type="hidden" name="_next" value="http://localhost:4200/kapcsolat">
      <div class="d-flex justify-content-center">
        <button class="mt-3 mb-1" type="submit" (click)="sendEmail()">Küldés</button>
      </div>
    </form>
  </div>
</div>

```

20. kép (Kapcsolat - Frontend)

Az form elküldéséhez a „formsubmit” nevű szolgáltatást használtuk, ez ingyenes és végtelen emailt továbbít az adott emailcímre. Megadtuk az input mezőket: Név, Email, Tárgy, illetve kapott két láthatatlan típusú input mezőt is, a „_captcha” névre hallgató mező kikapcsolja a az ellenőrzését a spameknek és robotoknak.

A második ilyen mező, a „_next” pedig az átirányításért felelős, mivel alaphoz a formsubmit oldalára irányít minket át sikeres üzenetküldés esetén. Ebben az esetben kapunk egy alertet elküldéskor, majd visszairányít minket a Kapcsolat komponensre.

Rólunk

Az oldalon található rólunk egy rövid leírás képekkel, a csapattagokról, magáról a webalkalmazásról.

```

1 <app-header></app-header>
2 <div class="container">
3   <div data-aos="fade-down" data-aos-duration="1500" class="section">
4     <p>
5       <b>Bemutatók: </b>
6       A CashTrack költségvetés-kezelő egy online weboldal, amely lehetővé teszi a felhasználók számára,
7       hogy egyszerűen kezeljék, nyomon kövessék és elemezzék pénzügyi kiadásait és bevételeit.
8       A weboldal célja, hogy segítse a felhasználókat a pénzügyeik átláthatóságának növelésében,
9       lehetővé téve számukra a költségvetésük részletes kezelését és esetleges rendszerezését.
10    </p>
11    
12  </div>
13  <div data-aos="fade-down" data-aos-duration="2000" class="section">
14    
15    <p>
16      <b>Csapatunk bemutatása:</b>
17      <ul>
18        <li>
19          <b>Martin:</b>
20          Martin egy 18-es skálán jó indulattal 8-asra értékelhető külsejű programozó.
21          Azon közeposztálybeli fehér európai férfiak közé tartozik akik élvezik az életet.
22          Néhány ember szerint Azahriah-ra hasonlít, nem lehet ellene érvelni.
23          Összességében elég nehéz rosszat mondani róla, szerethető figura.</li>
24        <li>
25          <b>Máté:</b>
26          Máté egy kreatív és céltudatos srác, aki szenvedélyesen szereti a technológiát és az innovációt.
27          Gyerekkora óta érdekli a programozás és a dizájn iránt, és mindig arra törekszik, hogy új dolgokat tanuljon és fejlessze önmagát.
28        </li>
29        <li>
30          <b>Dominik:</b>
31          Dominik egy lelkes és vidám fiatal, aki imádja a videojátékokat és a futballmeccsek izgalmát. Gyerekkora óta rajong a gaming világért,
32          és mindig nyitott az új kihívásokra, legyen szó stratégiáról, csapatjátékról vagy egy jó versengésről. Emellett szívesen követi kedvenc futballcsapatját,
33          és szenvedélyesen elemzi a játékokat, taktikákat.</li>
34      </ul>
35    </p>
36  </div>
37  <div data-aos="fade-down" data-aos-duration="2500" class="section">
38    <p><b>Céljaink a jövőben:</b></p>
39    <p>A weboldal célja, hogy felhasználóbarát és átlátható módon segítse az emberek személyes és üzleti pénzügyeinek kezelését.
40    A rendszer célja, hogy hozzájáruljon a tudatosabb pénzügyi döntésekhez, ezáltal segítve a felhasználókat abban, hogy hosszú távon stabilabb pénzügyi helyzetet érjenek el.</p>
41    
42  </div>

```

21. kép (Rólunk - Frontend)

Bejelentkezés

```

1 <app-header></app-header>
2 <div class="container bejelentkezes">
3   <div class="row">
4     <div class="col-3">
5       <!--BLANK-->
6     </div>
7     <div data-aos="fade-down" data-aos-duration="2500" class="col-6 login">
8       <div class="logo">
9         <div class="banner">
10          <div class="title">
11            <div class="desktop-logo">
12              <svg width="345" height="55" viewBox="0 0 345 55" fill="none" xmlns="http://www.w3.org/2000/svg">
13                <!--BLANK-->
14              </svg>
15            </div>
16            <div class="mobile-logo">
17              
18            </div>
19          </div>
20        </div>
21        <br>
22        <form (ngSubmit)="bejelentkezes()">
23          <h3> Bejelentkezés</h3>
24          <div class="mb-3">
25            <label class="form-label">E-mail</label>
26            <input type="email" [(ngModel)]="email" name="email" class="form-control" id="email" aria-describedby="emailHelp" placeholder="E-mail">
27          </div>
28          <div class="mb-3">
29            <label class="form-label">Jelszó</label>
30            <input type="password" [(ngModel)]="jelszo" name="jelszo" class="form-control" id="password" placeholder="Jelszó">
31          </div>
32          <div class="mb-3 centered">
33            <button type="submit" class="loginButton">Bejelentkezés</button>
34          </div>
35        </form>
36        <p class="question"><b>Nincs még fiókod?</b> <button class="btn btn-dark" (click)="registerRoute()">Regisztrálj!</button></p>
37      </div>
38    </div>
39  </div>
40  <div class="col-3">
41    <!--BLANK-->
42  </div>
43 </div>

```

22. kép (Bejelentkezés - Frontend)

Itt is megjelenik az svg kóddal megjelenített logó, ugyan azokkal a stíuselemekkel és animációval.

Amikor a felhasználó beírja az adatait, a bejelentkezes() függvény lefut, és ellenőrizteti a backenddel az adatokat.

```

35  belepes(): void {
36      if (!this.email || !this.jelszo) {
37          alert('Kérjük, töltsd ki az email és jelszó mezőket!');
38          return;
39      }
40      this.loginService.login(this.email, this.jelszo).subscribe({
41          next: (response: any) => {
42              if (response) {
43                  localStorage.setItem('felhasznalo', JSON.stringify(response));
44                  alert('Sikeres bejelentkezés!');
45                  this.authService.login();
46                  this.authService.isLoggedIn.set(true);
47                  this.kiadasService.kiadasokLekeres();
48                  this.jovedelemService.jovedelemLekeres();
49                  this.kiadasService.kiadasKategoriakLekeres();
50                  this.jovedelemService.kategoriakLekeres();
51                  setTimeout(() => {
52                      this.router.navigate(['/dashboard']);
53                  }, 2000);
54              }
55              else {
56                  console.log('Sikertelen bejelentkezés, hibás email vagy jelszó.');
57                  alert('Sikertelen bejelentkezés, hibás email vagy jelszó.');
58              }
59          },
60          error: (error) => {
61              console.error('Bejelentkezési hiba:', error);
62              alert('Hiba történt a bejelentkezés során. Kérjük próbálja újra.');
```

23. kép (Bejelentkezés - Frontend)

Ha sikeres, akkor lekéri az adatokat, illetve el is menti azokat a local storage-be. Eközben triggerelődik egy timeout, ami két másodperc múlva az irányítópultra navigálja a felhasználót, annak érdekében hogy biztosan megkapja az adatokat a böngésző.

Header

```

<div class="container">
  <div class="header mt-3 mb-5" id="header">
    <div class="logo-container col-5 col-lg-6 col-md-4 col-sm-5 inline">
      <div data-aos="fade-down" data-aos-duration="1500">
        <a routerLink="/" *ngIf="!loggedIn(); else dashPicture"></a>
      </div>
      <ng-template #dashPicture></ng-template>
    </div>

    <div class="nav-container">
      <div class="desktop-menu">
        <div data-aos="fade-down" data-aos-duration="2000">
          <a routerLink="/kapcsolat">Kapcsolat</a>
        </div>
        <div data-aos="fade-down" data-aos-duration="2500">
          <a routerLink="/rolunk">Rólunk</a>
        </div>
        <div data-aos="fade-down" data-aos-duration="3000">
          <a routerLink="/login" *ngIf="!loggedIn(); else dash">Bejelentkezés</a>
          <ng-template #dash data-aos="fade-down" data-aos-duration="3000">
            <a routerLink="/dashboard">Irányítópult</a>
          </ng-template>
        </div>
      </div>

      <button class="hamburger-menu" (click)="toggleMenu()">
        <i class="bi bi-list"></i>
      </button>

      <div class="mobile-menu" [class.active]="isMenuOpen">
        <a routerLink="/">Főoldal</a>
        <a routerLink="/kapcsolat">Kapcsolat</a>
        <a routerLink="/rolunk">Rólunk</a>
        <a routerLink="/login" *ngIf="!loggedIn(); else mobileDash">Bejelentkezés</a>
        <ng-template #mobileDash>
          <a routerLink="/dashboard">Irányítópult</a>
        </ng-template>
      </div>
    </div>
  </div>
</div>
```

24. kép (Header - Frontend)

A fejléc tartalmaz egy logót vagy dashboard linket a felhasználó bejelentkezési állapotától függően, valamint egy navigációs menüt az oldalakhoz. Az animációkat az AOS könyvtár kezeli, a gombnyomásra előugró hamburger menü pedig mobilnézetben jelenik meg. Bootstrap osztályokkal van reszponzívvá téve, és az *ngIf segítségével dinamikusan változik a megjelenő tartalom.

```
loggedIn = computed(() => this.authService.isLoggedIn());
isMenuOpen = false;

ngOnInit(): void {
  console.log('loggedIn', this.loggedIn());
  AOS.init();

  this.subscription = this.authService.isLoggedIn$.subscribe(() => {
    console.log('Bejelentkezési állapot változott:', this.loggedIn());
  });

  this.routerSubscription = this.router.events.pipe(
    filter(event => event instanceof NavigationEnd)
  ).subscribe(() => {
    console.log('Navigáció történt, bejelentkezési állapot frissítése');
    this.authService.updateLoginStatus();
  });
}

ngOnDestroy(): void {
  if (this.subscription) {
    this.subscription.unsubscribe();
  }
  if (this.routerSubscription) {
    this.routerSubscription.unsubscribe();
  }
}

toggleMenu(): void {
  this.isMenuOpen = !this.isMenuOpen;
}
}
```

25. kép (Header - Frontend)

Itt figyeljük a bejelentkezési állapotot és az oldalon belüli navigációkat. Ha a bejelentkezési állapot megváltozik vagy egy új oldalra lépünk, frissíti az állapotot és üzenetet ír ki a konzolra.

Register

```
<h3>Regisztráció</h3>
<div class="mb-3">
  <label class="form-label toLeft">Vezetéknév</label>
  <input type="email" class="form-control" [(ngModel)]="vezeteknev" id="vezeteknev" aria-describedby="emailHelp" name="vezeteknev" placeholder="pl. Minta">
</div>
<div class="mb-3">
  <label class="form-label toLeft">Keresztnev</label>
  <input type="email" class="form-control" [(ngModel)]="keresztnev" id="keresztnev" aria-describedby="emailHelp" name="keresztnev" placeholder="pl. Ernő">
</div>
<div class="mb-3">
  <label class="form-label toLeft">E-mail</label>
  <input type="email" class="form-control" [(ngModel)]="emailcim" id="email" aria-describedby="emailHelp" name="email" placeholder="pl. erno@cashtrack.hu">
</div>
<div class="mb-3">
  <label for="exampleInputPassword1" class="form-label toLeft">Jelszó</label>
  <input type="password" [(ngModel)]="password" class="form-control" id="password" name="password" placeholder="Jelszó">
</div>
<div class="mb-3 mt-3 centered">
  <button type="submit" class="loginButton">Regisztráció</button>
</div>
</form>
<p class="question"><b>Már van fiókod?</b> &nbsp;<button class="btn btn-dark" (click)="LoginRoute()">Jelentkezz be!</button></p>
</div>
```

26. kép (Regisztráció – Frontend)

```
vezeteknev: string = '';
keresztnev: string = '';
emailcim: string = '';
password: string = '';
regisztracio(): void {
  console.log('onsubmit fuggveny');
  const userData = { vezeteknev: this.vezeteknev,
    keresztnev: this.keresztnev,
    email: this.emailcim,
    jelszo: this.password };
  this.regiserService.registerUser(userData).subscribe((response:any)=>{
    console.log(response);
    if(response){
      alert('Sikeres regisztráció');
      this.router.navigate(['/login']);
    }
    else{
      alert('Sikertelen regisztráció');
    }
  });
}

LoginRoute(): void {
  this.router.navigate(['/login']);
}
```

27. kép (Regisztráció – Frontend)

A weboldalon történő regisztráció során bekérjük a felhasználó nevét, e-mail címét és jelszavát, hogy személyre szabott fiókot tudjunk létrehozni.

Dashboard

```


<div class="modal-dialog">
  <div class="modal-content">
    <div class="modal-header">
      <h1 class="modal-title fs-5" id="exampleModalLabel">Bevétel hozzáadása/h1>
      <button type="button" class="btn-close btn-light" data-bs-dismiss="modal" aria-label="Close"></button>
    </div>
    <div class="modal-body">
      <p style="display: flex; justify-content: space-between;">
        <span class="bi bi-list-check text-light">&nbsp;&nbsp;&nbsp;Bevétel típusa: </span>
        <select name="expenseCategory" id="incomeCategory" [(ngModel)]="jovedelemTpus">
          <option value="Választ" Válassz!</option>
          <option *ngFor="let kategoria of jovedelemKategoriatomb" [value]="kategoria.kategoriaID">{{kategoria.jovedelemKategoria}}</option>
        </select>
      </p>
      <p style="display: flex; justify-content: space-between;">
        <span class="bi bi-cash text-light">&nbsp;&nbsp;&nbsp;Érték: </span>
        <input type="number" name="incomeEUF" id="incomeEUF" placeholder="Érték forintban" [(ngModel)]="jovedelem">
      </p>
      <p style="display: flex; justify-content: space-between;">
        <span class="bi bi-calendar-date text-light">&nbsp;&nbsp;&nbsp;Bevétel dátuma: </span>
        <input type="date" name="incomeDate" id="incomeDate" [(ngModel)]="jovedelemDatum"></p>
    </div>
    <div class="modal-footer">
      <button type="button" class="btn btn-danger" data-bs-dismiss="modal">Mégsem</button>
      <button type="button" class="btn btn-success" (click)="jovedelemHozzaadas()" data-bs-dismiss="modal">Bevétel mentése</button>
    </div>
  </div>
</div>
</div>
</div>

<div class="modal fade expenseModal" id="addExpense" tabindex="-1" aria-labelledby="exampleModalLabel" aria-hidden="true">
<div class="modal-dialog">
  <div class="modal-content">
    <div class="modal-header">
      <h1 class="modal-title fs-5" id="exampleModalLabel">Kiadás hozzáadása/h1>
      <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
    </div>
    <div class="modal-body">
      <form>
      <div class="modal-body">
        <p style="display: flex; justify-content: space-between;">
          <span class="bi bi-list-check text-light">&nbsp;&nbsp;&nbsp;Kiadás típusa: </span>
          <select name="expenseCategory" id="expenseCategory" [(ngModel)]="kiadasTpus">
            <option value="Választ" Válassz!</option>
            <option *ngFor="let kategoria of kiadasKategoriatomb" [value]="kategoria.kategoriaID">{{ kategoria.kiadasKategoria }}</option>
          </select></p>
        <p style="display: flex; justify-content: space-between;">
          <span class="bi bi-cash text-light">&nbsp;&nbsp;&nbsp;Érték: </span>
          <input type="number" name="expenseEUF" id="expenseEUF" [(ngModel)]="kiadasErtek" placeholder="Érték for">
        </p>
        <p style="display: flex; justify-content: space-between;">
          <span class="bi bi-calendar-date text-light">&nbsp;&nbsp;&nbsp;Kiadás dátuma: </span>
          <input type="date" name="expenseDate" id="expenseDate" [(ngModel)]="kiadasDatum"></p>
        <textarea name="expenseComment" id="expenseComment" [(ngModel)]="kiadasMegjegyzes" placeholder="pl. Vásárlás során választott termékek - NEH KÖTELEZŐ!"></textarea>
      </div>
      <div class="modal-footer">
        <button type="button" class="btn btn-danger" data-bs-dismiss="modal">Mégsem</button>
        <button type="button" class="btn btn-success" data-bs-dismiss="modal" (click)="kiadasHozzaadas()">Kiadás mentése</button>
      </div>
    </div>
  </div>
</div>
</div>
</div>


```

28. kép (Irányítópult – Frontend)

```
<div class="modal fade" id="jovedelemModal" tabindex="-1" aria-labelledby="exampleModallabel" aria-hidden="true">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <h1 class="modal-title fs-5" id="exampleModallabel">Összes jövedelem</h1>
        <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close" (click)="ngOnInit()" ></button>
      </div>
      <div class="modal-body">
        <app-incomelist></app-incomelist>
      </div>
    </div>
  </div>
</div>

<div class="modal fade" id="kiadasokModal" tabindex="-1" aria-labelledby="exampleModallabel" aria-hidden="true">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <h1 class="modal-title fs-5" id="exampleModallabel">Összes kiadás</h1>
        <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close" (click)="ngOnInit()" ></button>
      </div>
      <div class="modal-body">
        <app-expenselist></app-expenselist>
      </div>
    </div>
  </div>
</div>
```

29. kép (Irányítópult – Frontend)

Itt egy helyen látjuk az aktuális egyenlegünket, a bevételeink és kiadásaink összegzését, valamint kategóriák szerinti lebontását is, például étkezésre, rezsire vagy szórakozásra.


```

jovedelemHaviTemp : number = 0;
havijovedelmek = computed(() => this.jovedelemService.szamolas());
HaviJovedelemFrissitese(){
  this.jovedelemService.jovOsszeadas()
  console.log('havijovdelemek signal',this.havijovedelmek())
}

havikiadasokSzamolo:number = 0;
havikiadasok =computed(() => this.kiadasService.szamolas());

HaviKiadasokFrissitese(){
  this.kiadasService.kiadOsszeadas()
}

haviosszes = computed(() => this.havijovedelmek() - this.havikiadasok()); ;

HaviOsszesFrissitese(){
}

logout(): void {
  this.authService.logout();
  this.router.navigate(['/home']);
  localStorage.clear();
}

notWorking(): void{
  alert('Ez a funkció jelenleg nem elérhető!');
}

ngOnDestroy(): void {
  if (this.subscription) {
    this.subscription.unsubscribe();
  }
}
}

```

30. kép (Irányítópult – Frontend)

Itt kezeljük a kiadásokat és a bevételeket havi szinten amiket a felhasználó megad.

Piechart

```

<canvas baseChart
  [type]=" 'pie' "
  [datasets]="pieChartDatasets"
  [labels]="pieChartLabels"
  [options]="pieChartOptions"
  [plugins]="pieChartPlugins"
  [legend]="pieChartLegend">
</canvas>

```

```

refreshChart() {
  setTimeout(() => {

    const jovedelmek= this.jovedelemtomb();
    console.log('refreshChart metodus',this.jovedelemtomb());

    if (!jovedelmek || jovedelmek.length === 0) {
      console.log("Nincsenek adatok a charthoz.");
      return;
    }

    const categoryCounts: { [key: string]: number } = {};
    jovedelmek.forEach(jovedelem => {
      if (jovedelem.kategoria && jovedelem.kategoria.jovedelemKategoria) {
        const osszeg = jovedelem.kategoria.jovedelemKategoria;
        categoryCounts[osszeg] = (categoryCounts[osszeg] || 0) + jovedelem.bevetelHUF;
      }
    });

    if (Object.keys(categoryCounts).length === 0) {
      console.warn("Nem található érvényes kategória a chart frissítéséhez.");
      return;
    }

    this.pieChartLabels = Object.keys(categoryCounts);
    this.pieChartDatasets[0].data = Object.values(categoryCounts);

    if (!this.chart) {
      console.warn("A chart még nem inicializálódott.");
      return;
    }

    this.chart.update(); // Frissítés
  }, 2000);
}

```

31. kép, 32. kép (Piechart – Frontend)

Az npm2chart segítségével lett használva. A komponens kizárólag a grafikont tartalmazza.

Modellek

3 Modellünk van

Felhasználó, Jovedelem, Kiadás:

```
export interface Felhasznalo
{
    felhasznaloID:number;
    vezeteknev:string;
    keresztnév:string;
    email:string;
    jelszo:string;
    profilkepUrl:string;
}

export interface Jovedelem {
    jovedelemID?: number;
    felhasznaloID: number;
    bevetelHUF: number;
    bevetelDatum: string;
    kategoriaID: any;
    kategoria?: any[];
}

export interface Kiadas
{
    kiadasID?:number;
    felhasznaloID:number;
    kiadasHUF:number;
    kiadasDatum:string;
    kategoriaID: any;
    kiadasKomment:string;
    kategoria?: any[];
}
```

33. kép, 34. kép, 35. kép (Modellek – Frontend)

Az adatokat interfészekben tároljuk az alkalmazás számára. Ezek az interfészek meghatározzák az alkalmazásban használt adatok típusait.

Service-k

```
private felhasznaloUrl = 'http://127.0.0.1:8000/api/felhasznalok';
private felhasznaloEmailUrl = 'http://127.0.0.1:8000/api/felhasznalok/';
constructor(private http: HttpClient) { }

getFelhasznalok(): Observable<any> {
    return this.http.get(this.felhasznaloUrl);
}

getFelhasznaloByEmail(): Observable<any>{
    return this.http.get(this.felhasznaloEmailUrl);
}

registerUser(userData: { vezeteknev: string, keresztnév: string, email: string, jelszo: string }): Observable<any> {
    return this.http.post(this.felhasznaloUrl, userData, {
        headers: new HttpHeaders({
            'Content-Type': 'application/json'
        })
    });
}
```

36. kép (Service – Frontend)

Ez a register service, A **registerUser** metódus új felhasználót regisztrál egy POST kérés segítségével, ennek a service-nek ez a fő feladata.

```

private apiUrl = 'http://127.0.0.1:8000/api/felhasznalok';

constructor(private http: HttpClient) { }

login(email: string, jelszo: string): Observable<any> {
  return this.http.get(`${this.apiUrl}/email/${email}`).pipe(
    map((response: any) => {
      if (response && response.jelszo === jelszo) {
        return response;
      }
      return null;
    })
  );
}

logout(): void {
  localStorage.removeItem('felhasznalo');
}
}

```

Ez a login service, Ez a szerviz egy felhasználó bejelentkezését és kijelentkezését kezeli.

```

private apiUrlJovedelemKategoriak = 'http://127.0.0.1:8000/api/jovedelemkategoriak';
private apiUrlJovedelem = 'http://127.0.0.1:8000/api/jovedelem';
constructor(private http: HttpClient) {}

jovedelemAdat = signal<Jovedelem[]>([]);
jovedelemKategoriak: any[] = [];
jovedelem: {jovedelemID: number, felhasznaloID: number, bevetelHUF: number, bevetelDatum: string, kategorialID: any }[] = [];

Kategorialekkereso() {
  this.http.get(`${this.apiUrlJovedelemKategoriak}`).subscribe((data: any) => {
    this.jovedelemKategoriak = data;
    localStorage.setItem('jovedelemkategoriak', JSON.stringify(this.jovedelemKategoriak));
  });
}

jovedelemekkereso() {
  const user = JSON.parse(localStorage.getItem('felhasznalo')) || '{}';
  this.http.get(`${this.apiUrlJovedelem}/felhasznalo/${user.felhasznaloID}`).subscribe((data: any) => {
    this.jovedelem = data;
    console.log(this.jovedelem);
    this.jovedelem.sort((a, b) => new Date(b.bevetelDatum).getTime() - new Date(a.bevetelDatum).getTime());
    localStorage.setItem('jovedelem', JSON.stringify(this.jovedelem));
    this.jovedelemAdat.set(this.jovedelem);
  });
}

jovedelemAdatok: Jovedelem[] = [];
jovedelemekkeresoJSON() {
  const user = JSON.parse(localStorage.getItem('felhasznalo')) || '{}';
  this.http.get(`${this.apiUrlJovedelem}/felhasznalo/${user.felhasznaloID}`).subscribe((data: any) => {
    this.jovedelemAdatok = data;
    this.jovedelemAdatok.sort((a, b) => new Date(b.bevetelDatum).getTime() - new Date(a.bevetelDatum).getTime());
  });
  return this.jovedelemAdatok;
}

private jovedelemkulcs = "jovedelem";

private jovedelemVizsgalatiKereso(): any[] {
  return JSON.parse(localStorage.getItem(this.jovedelemkulcs)) || '[]';
}

jovedelemVizsgalatiKeresoJSON(): Jovedelem[] {
  return this.jovedelemVizsgalatiKereso();
}

jovedelemFeltoltes(jovedelemAdatok: Jovedelem[]): Observable<any> {
  return this.http.post(`${this.apiUrlJovedelem}`, jovedelemAdatok);
}

jovedelemekFrissitese(ujJovedelem: any[]) {
  localStorage.setItem(this.jovedelemkulcs, JSON.stringify(ujJovedelem));
  this.jovedelemAdat.update(jovedelemAdat => [...jovedelemAdat, ...ujJovedelem]);
}

jovedelemHozzaadas(jovedelemAdat: {felhasznaloID: number, bevetelHUF: number, bevetelDatum: string, kategorialID: any}): Observable<any> {
  const frissítettAdat = [...this.jovedelemVizsgalatiKereso(), jovedelemAdat];
  this.jovedelemFrissitese(frissítettAdat);
  return this.http.post(`${this.apiUrlJovedelem}`, jovedelemAdat);
}

}

tomb: any[] = [];
szamolas = signal<number>(0);
countTemp = 0;
jovosszadas() {
  const user = JSON.parse(localStorage.getItem('felhasznalo')) || '{}';
  this.http.get(`${this.apiUrlJovedelem}/felhasznalo/${user.felhasznaloID}`).subscribe((data: any) => {
    this.tomb = data;
    console.log('Tomb adatok (jovedelem)', this.tomb);
    this.countTemp = 0;
    this.tomb.forEach(element => {
      this.countTemp += element.bevetelHUF;
      console.log('countTemp változó:', this.countTemp);
    });
    this.szamolas.update(count => this.countTemp);
    console.log('Ez itt mi?', this.szamolas());
  });
  return this.szamolas;
}

```

Ez a szerviz a jövedelem kategóriák és azok adatai kezelésére szolgál.

```
kiadaskategoriatomb:any[] = []
kiadasoklekereke(){
  const user = JSON.parse(localStorage.getItem("felhasznalo")) || "{}";
  this.http.get(`${this.apiUrl}/felhasznalo/${user.felhasznaloID}`).subscribe((data:any) => {
    this.kiadasokOsszes = data;
    this.kiadasokOsszes.sort((a, b) => new Date(b.kiadasDatum).getTime() - new Date(a.kiadasDatum).getTime());
    localStorage.setItem('kiadasok', JSON.stringify(this.kiadasokOsszes));
    this.kiadasAdat.set(this.kiadasokOsszes);
  })
}

kiadaskategorialekerese(): any[] {
  this.http.get(`${this.apiUrl}/kiadaskategoriak`).subscribe((data:any) => {
    this.kiadaskategoriatomb = data;
    localStorage.setItem('kiadaskategoriak', JSON.stringify(this.kiadaskategoriatomb));
  })
  return this.kiadaskategoriatomb;
}

kiadasTorles(kiadasID: number):Observable<any>{
  let taroltomb = JSON.parse(localStorage.getItem("kiadasok")) || "[]";
  taroltomb = taroltomb.filter((item: any) => item.kiadasID !== kiadasID);
  localStorage.setItem('kiadasok', JSON.stringify(taroltomb));
  console.log(JSON.parse(localStorage.getItem("kiadasok")) || "[]");
  return this.http.delete(`${this.apiUrl}/${kiadasID}`);
}

private kiadaskulcs = "kiadasok";
private kiadasokFigyeles = new BehaviorSubject<any>((this.kiadasoklekerekeReturn()));
kiadasok$ = this.kiadasokFigyeles.asObservable();
private kiadasoklekerekeReturn(): any[] {
  return JSON.parse(localStorage.getItem(this.kiadaskulcs)) || "[]";
}
kiadasokReturn(): any[] {
  return this.kiadasoklekerekeReturn();
}

kiadasTorles(index: number, kiadasID: number) {
  const frissitettKiadas = this.kiadasoklekerekeReturn().filter((_, i) => i !== index);
  this.kiadasokFrissitese(frissitettKiadas);
  console.log(JSON.parse(localStorage.getItem(this.kiadaskulcs)) || "[]");
  return this.http.delete(`${this.apiUrl}/${kiadasID}`).subscribe(Response => {
    console.log(Response);
    this.kiadOsszeadas();
  });
}

kiadasokFrissitese(ujKiadasok: Kiadas[]) {
  localStorage.setItem(this.kiadaskulcs, JSON.stringify(ujKiadasok));
  this.kiadasokFigyeles.next(ujKiadasok);
  this.kiadasAdat.update(kiadasok => [...kiadasok, ...ujKiadasok]);
  console.log('Service Signal Frissitve frissitett', this.kiadasAdat());
}

kiadasHozzaadas(kiadasAdat: Kiadas):Observable<any>{
  const frissitettAdat = [...this.kiadasoklekerekeReturn(), kiadasAdat];
  this.kiadasokFrissitese(frissitettAdat);
  return this.http.post(`${this.apiUrl}`, kiadasAdat)
}

tomb:any[] = [];
countTemp = 0;
szamolas = signal<number>(0);
kiadoOsszeadas(){
  const user = JSON.parse(localStorage.getItem("felhasznalo")) || "{}";
  this.http.get(`${this.apiUrl}/felhasznalo/${user.felhasznaloID}`).subscribe((data:any) => {
    this.tomb = data;
    console.log('Tomb adatok (jövedelem)',this.tomb);
    this.countTemp = 0;
    this.tomb.forEach(element => {
      this.countTemp += element.kiadasMF;
      console.log("countTemp változó:",this.countTemp);
    });
    this.szamolas.update(count => this.countTemp)
    console.log("Ez itt a1?",this.szamolas());
  });
  return this.szamolas;
}
```

Ez a szerviz a kiadás kategóriák és azok adatai kezelésére szolgál.

Reszponzivitás

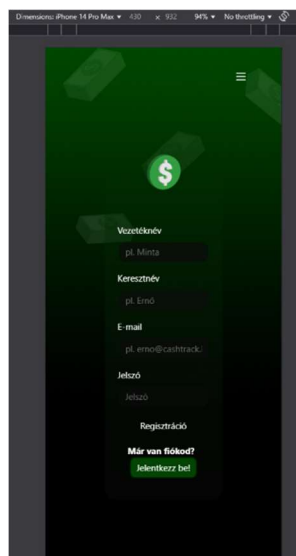
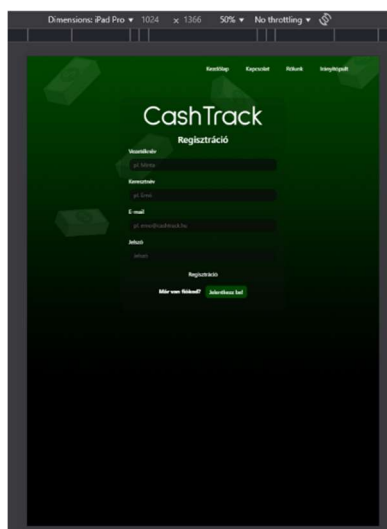
A részponzivitást egy iPhone 14 Pro Maxon, illetve egy iPad Pron teszteltük.

Kezdőlap

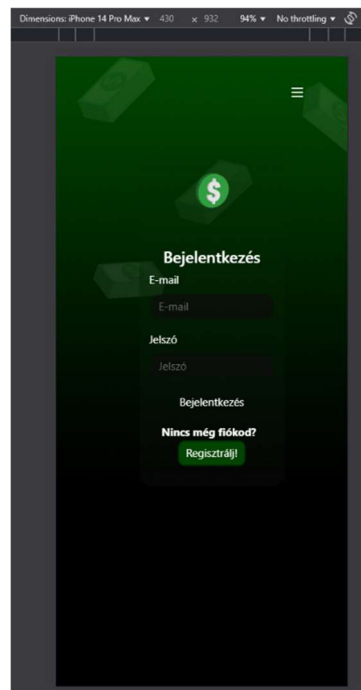
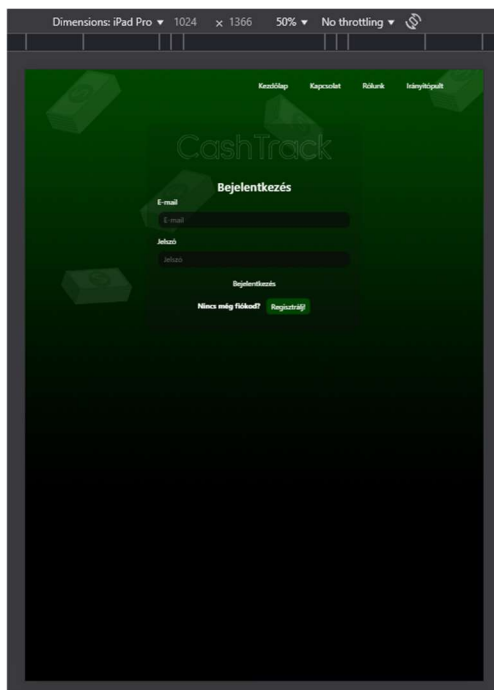


37. kép, 38. kép (Reszponzivitás – Frontend)

Bejelentkezés és Regisztráció



39. kép, 40. kép (Reszponzivitás – Frontend)



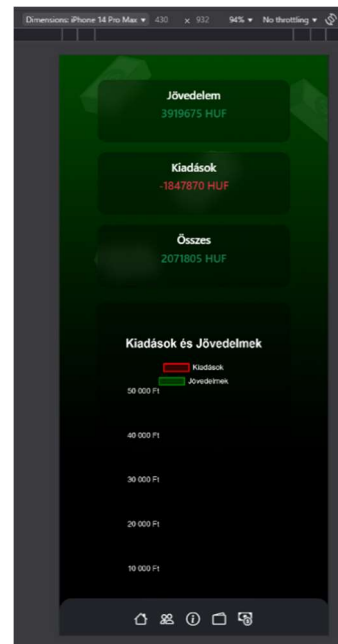
41. kép, 42. kép (Reszponzivitás – Frontend)

Rólunk



41. kép, 42. kép (Reszponzivitás – Frontend)

Irányítópult



43. kép, 44. kép (Reszponzivitás – Frontend)

Backend

Alkalmazás backend részének részletes leírása

Dokumentációnk ezen része a CashTrack webalkalmazás fejlesztői oldalát részletesen mutatja be.

API Végpontok

Az API egy közvetítő, amely összeköti az alkalmazásokat, például egy frontend felületet a backenddel, és biztosítja, hogy ezek hatékonyan kommunikáljanak egymással.

Fontos: Az API végpontok teszteléséhez a **Thunder Client** használata javasolt a Visual Studio Code-on belül.

Thunder Client telepítése

1. Nyissuk meg a Visual Studio Code-ot.
2. Lépünk az 'Extensions' (Bővítmények) fülre az oldalsávon.
3. A keresőbe írjuk be a következőt: **Thunder Client**.
4. Telepítsük a bővítményt.
5. Ha jól dolgoztunk, egy villám ikon fog az oldalsávon megjelenni.

”Felhasználó” lekérdezések

Route	Lekérdezés típusa	Rövid leírás
/felhasznalok	GET	Összes felhasználó lekérdezése.

/felhasznalok/{felhasznaloID}	GET	Specifikus felhasználó lekérdezése ID alapján.
/felhasznalok/email/{email}	GET	Specifikus felhasználó lekérdezése e-mail alapján.
/felhasznalok	POST	Felhasználó feltöltése.
/felhasznalok/{felhasznaloID}	PUT	Specifikus felhasználó adatainak frissítése.
/felhasznalok/{felhasznaloID}	DELETE	Specifikus felhasználó törlése.

”Jövedelem” lekérdezések

Route	Lekérdezés típusa	Rövid leírás
/jovedelmek	GET	Összes jövedelem lekérdezése.
/jovedelmek	POST	Jövedelem feltöltése.
/jovedelmek/{jovedelemID}	GET	Specifikus jövedelem lekérdezése ID alapján.
/jovedelmek/{jovedelemID}	DELETE	Jövedelem törlése.

”Jövedelem kategóriák” lekérdezések

Route	Lekérdezés típusa	Rövid leírás
/jovedelemkategoriak	GET	Összes jövedelemkategória lekérdezése.

”Kiadás” lekérdezések

Route	Lekérdezés típusa	Rövid leírás
/kiadasok	GET	Összes kiadás lekérdezése.
/kiadasok	POST	Kiadás feltöltése.
/kiadasok/{kiadasID}	GET	Specifikus kiadás lekérdezése ID alapján.
/kiadasok/felhasznalo/{felhasznaloid}	GET	Specifikus felhasználó kiadásainak lekérdezése.
/kiadasok/{kiadasID}	PUT	Specifikus kiadás frissítése.
/kiadasok/{kiadasID}	DELETE	Specifikus kiadás törlése.

”Kiadás kategóriák” lekérdezések

Route	Lekérdezés típusa	Rövid leírás
/kiadaskategoriak	GET	Összes kiadás kategória lekérdezése.

Hibák hibás lekérdezésekkor

Hibakód	Üzenet	Hiba
---------	--------	------

404	„Felhasználó nem található”	A felhasználó nem található az adatbázisban.
400	„Nem megfelelő adatok”	Hibásan adtuk meg az adatokat feltöltéskor vagy frissítéskor.
400	„Nem található kiadás”	Nem létezik az adott kiadás az adatbázisban.
404	„Jövedelem nem található”	Nem létezik az adott jövedelem az adatbázisban.

Kontrollerek

Egy backend kontroller felelős a beérkező HTTP kérések feldolgozásáért. Fogadja a kéréseket, végrehajtja a szükséges üzleti logikát, és válaszokat küld vissza a felhasználónak. A kontroller közvetíti az adatokat az alkalmazás más rétegei (pl. adatbázis) és a frontend között.

A CashTrack öt kontrollerrel rendelkezik: FelhasznaloController, JovedelemController, JovedelemKategoriakController, KiadasController, KiadasKategoriakController.

FelhasznaloController

A FelhasznaloController feladata, hogy a frontend kérésére reagáljon és megfelelő válaszokat küldjön a felhasználói műveletekkel kapcsolatban. Az adatokat az Felhasznalo modellel kezeli, és a válaszokat JSON formátumban adja vissza.^{45. kép (Kontrollerek - Backend)}

Az alábbi funkciókat hajtja végre:

```
public function index()
{
    return Felhasznalo::all();
}
```

index(): Visszaadja az összes felhasználót.

```
public function getFelhasznalo()
{
    return response()->json(Felhasznalo::all(),200);
}
```

46. kép (Kontrollerek - Backend)

getFelhasznalo(): JSON formátumban visszaadja az összes felhasználót.

```
public function getFelhasznaloById($felhasznaloID){
    $felhasznalo = Felhasznalo::find($felhasznaloID);
    if(is_null($felhasznalo)){
        return response()->json(['message' => 'Felhasznalo nem talalhato'], 404);
    }
    return response()->json($felhasznalo, 200);
}
```

47. kép (Kontrollerek - Backend)

getFelhasznaloById(\$felhasznaloID): Megkeresi a felhasználót az ID alapján, és ha nem található, 404-es hibaüzenetet ad vissza.

```
public function getFelhasznaloByEmail(string $email)
{
    $felhasznalo = Felhasznalo::where('email', $email)->first();

    if (is_null($felhasznalo)) {
        return response()->json(['message' => 'Felhasznalo nem talalhato'], 404);
    }

    return response()->json($felhasznalo, 200);
}
```

48. kép (Kontrollerek - Backend)

getFelhasznaloByEmail(\$email): Megkeresi a felhasználót az e-mail cím alapján, és ha nem található, 404-es hibaüzenetet ad vissza.

```
public function addFelhasznalo(request $request){
    $validator = Validator::make($request->all(),[
        'vezeteknev' => 'required',
        'keresztnev' => 'required',
        'email' => 'required',
        'jelszo' => 'required'
    ]);
    if($validator->fails()){
        return response()->json($validator->errors(),400);
    }
    $felhasznalok = Felhasznalo::create($request->all());
    return response($felhasznalok,201);
}
```

49. kép (Kontrollerek - Backend)

addFelhasznalo(Request \$request): Felhasználót ad hozzá a rendszerhez. Ellenőrzi a bemeneti

adatokat (vezetéknév, keresztnév, email, jelszó), és ha a validáció nem sikerül, 400-as hibát ad vissza.

```
public function updateFelhasznalo(Request $request, $felhasznaloID){
    $felhasznalo = Felhasznalo::find($felhasznaloID);
    $validator = Validator::make($felhasznalo->all(),[
        'vezeteknev' => 'required',
        'keresztnev' => 'required',
        'email' => 'required',
        'jelszo' => 'required'
    ]);
    if(is_null($felhasznalo)){
        return response()->json(['message' => 'Felhasznalo nem talalhato'], 404);
    }
    $felhasznalo->update($request->all());
    return response($felhasznalo,200);
}
```

50. kép (Kontrollerek - Backend)

updateFelhasznalo(Request \$request, \$felhasznaloID): Frissíti egy meglévő felhasználó adatait, és ha nem található, 404-es hibaüzenetet ad vissza.

```
public function deleteFelhasznalo($felhasznaloID){
    $felhasznalo = Felhasznalo::find($felhasznaloID);
    if(is_null($felhasznalo)){
        return response()->json(['message' => 'Felhasznalo nem talalhato'], 404);
    }
    $felhasznalo->delete();
    return response()->json(null,204);
}
```

51. kép (Kontrollerek - Backend)

deleteFelhasznalo(\$felhasznaloID): Törli a felhasználót az ID alapján, és ha nem található, 404-es hibaüzenetet ad vissza.

JovedelemController

A **JovedelemController** a bevételek kezeléséért felelős. Az alábbi funkciókat hajtja végre:

```
public function index()
{
    return Jovedelem::all();
}
```

52. kép (Kontrollerek - Backend)

index(): Visszaadja az összes bevételt (Jovedelem).

```

public function store(Request $request)
{
    $validator = Validator::make($request->all(), [
        'felhasznaloID' => 'required',
        'bevetelHUF' => 'required',
        'bevetelDatum' => 'required',
        'kategoriaID' => 'required'
    ]);
    if($validator->fails()){
        return response()->json(['message' => 'Jovedelem nem található'],
            400);
    }
    $jovedelem = Jovedelem::create($request->all());
    return response()->json($jovedelem,201);
}

```

53. kép (Kontrollerek - Backend)

store(Request \$request): Új bevételt hoz létre. Ellenőrzi, hogy a szükséges mezők (felhasználóID, bevételHUF, bevétel dátuma, kategóriaID) rendelkezésre állnak-e, és ha nem, 400-as hibát ad vissza. Ha minden rendben van, a bevétel létrejön, és 201-es választ ad vissza.

```

public function show(Jovedelem $jovedelem,$jovedelemID)
{
    $jovedelem = Jovedelem::find($jovedelemID);
    if(is_null($jovedelem)){
        return response()->json(['message' => 'Felhasznalo nem található'],
            404);
    }
    return response()->json($jovedelem,200);
}

```

54. kép (Kontrollerek - Backend)

show(Jovedelem \$jovedelem, \$jovedelemID): Visszaadja a bevételt az adott ID alapján. Ha nem található, 404-es hibaüzenetet ad vissza.

```

public function showByUser(Request $request)
{
    $jovedelem = Jovedelem::where('felhasznaloID',$request->felhasznaloID)
->with('kategoria')->get();
    if(is_null($jovedelem)){
        return response()->json(['message' => 'Nem található a kiadás'],400);
    }
    else
        return response()->json($jovedelem,200);
}

```

55. kép (Kontrollerek - Backend)

showByUser(Request \$request): Visszaadja az adott felhasználóhoz tartozó bevételeket. A válasz tartalmazza a kapcsolódó kategóriát is. Ha nincs adat, 400-as hibát küld vissza.

```

public function update(Request $request, Jovedelem $jovedelem, $jovedelemID)
{
    $jovedelem = Jovedelem::find($jovedelemID);
    if(is_null($jovedelem)){
        return response()->json(['message' => 'Jovedelem nem talalhato'],
            404);
    }
    $jovedelem->update($request->all());
    return response($jovedelem,200);
}

```

56. kép (Kontrollerek - Backend)

update(Request \$request, Jovedelem \$jovedelem, \$jovedelemID): Frissíti az adott bevételt. Ha a bevétel nem található, 404-es hibaüzenetet ad vissza, egyébként frissíti az adatokat és 200-as választ küld.

```

public function destroy(Jovedelem $jovedelem,$jovedelemID)
{
    $jovedelem = Jovedelem::find($jovedelemID);
    if(is_null($jovedelem)){
        return response()->json(['message' => 'Jovedelem nem talalhato'],
            404);
    }
    $jovedelem->delete();
    return response()->json(null,204);
}

```

57. kép (Kontrollerek - Backend)

destroy(Jovedelem \$jovedelem, \$jovedelemID): Törli az adott bevételt. Ha nem található, 404-es hibaüzenetet ad vissza, egyébként törli az adatokat és 204-es választ ad.

JovedelemKategoriakController

A JovedelemKategoriakController a bevétel kategóriák kezeléséért felelős.

Az alábbi feladatokat hajtja végre:

```

public function index()
{
    return Jovedelem_kategoria::all();
}

```

58. kép (Kontrollerek - Backend)

index(): Visszaadja az összes jövedelem kategóriát.


```

public function store(Request $request)
{
    $validator = Validator::make($request->all(), [
        'jovedelemKategoria' => 'required'
    ]);
    if($validator->fails()){
        return response()->json(['message' => 'Nem megfelelő adatok'],400);
    }
    $kategoria = Jovedelem::create($request->all());
    return response($kategoria,201);
}

```

59. kép (Kontrollerek - Backend)

store(Request \$request): Új bevétel kategóriát hoz létre. A kérés **érvényesítése** történik a **jovedelemKategoria** mezőre, és ha az nem megfelelő, 400-as hibát ad vissza. Ha a validáció **sikeres**, létrehozza az új kategóriát és 201-es státusz kóddal visszaadja. Jelenleg nincs használatban ez a metódus.

```

public function show(jovedelemKategoria $jovedelemKategoria)
{
    $jovedelem = Jovedelem::find($kategoriaID);
    if(is_null($jovedelem)){
        return response()->json(['message' => 'Jovedelem nem található'],404);
    }
    return response()->json($jovedelem,200);
}

```

60. kép (Kontrollerek - Backend)

show(\$kategoriaID): Megkeresi a bevétel kategóriát ID alapján. Ha a kategória nem található 404-es hibakódot ad vissza, ha található akkor egy 200-as státusz kód mellé, JSON formátumban visszaadja az értéket.

A kontroller rendelkezik update, illetve destroy metódusokkal, de jelenleg nem üzemelnek, mivel meghatározott kategóriákkal dolgozunk.

KiadasController

A KiadasController a **Kiadas** modellel kapcsolatos műveletekért felelős.

```

public function index()
{
    return Kiadas::all();
}

```

61. kép (Kontrollerek - Backend)

index(): Visszaadja az összes adatbázisba rögzített kiadást JSON formátumban.

```
public function store(Request $request)
{
    $validator = Validator::make($request->all(), [
        'felhasznaloID' => 'required',
        'kiadasHUF' => 'required',
        'kiadasDatum' => 'required',
        'kategoriaID' => 'required'
    ]);
    if($validator->fails()){
        return response()->json(['message' => 'Nem megfelelő adatok'],400);
    }
    $kiadas = Kiadas::create($request->all());
    return response()->json($kiadas->all(),201);
}
```

62. kép (Kontrollerek - Backend)

store(Request \$request): Új **Kiadas** rekordot hoz létre a megadott adatokkal. Először validálja a bejövő adatokat (**felhasznaloID**, **kiadasHUF**, **kiadasDatum**, **kategoriaID**). Ha a validáció nem sikerül, 400-as hibát ad vissza a "Nem megfelelő adatok" üzenettel. Ha a validáció sikeres, létrehozza az új kiadást és 201-es válaszkóddal visszaadja az új rekordot.

```
public function update(Request $request, Kiadas $kiadas)
{
    $validator = Validator::make($request->all(), [
        'felhasznaloID' => 'required',
        'kiadasHUF' => 'required',
        'kiadasDatum' => 'required',
        'kategoriaID' => 'required'
    ]);
    if($validator->fails())
    {
        return response()->json(['message' => 'Nem megfelelő adatok'],400);
    }
    else{
        $kiadas = Kiadas::find($request->kiadasID);
        if(is_null($kiadas))
        {
            return response()->json(['message' => 'Nem található a kiadás'],400);
        }
        else
        {
            $kiadas->update($request->all());
            return response()->json($kiadas,200);
        }
    }
}
```

63. kép (Kontrollerek - Backend)

update(Request \$request, Kiadas \$kiadas): Frissíti a meglévő kiadást a megadott **Kiadas** rekord alapján. A bejövő adatokat validálja ugyanúgy, mint a **store** metódusban. Ha a validáció

nem sikerül, 400-as hibát ad vissza. Ha a kiadás nem található, 400-as hibát ad vissza a "Nem található a kiadás" üzenettel. Ha minden sikeres, frissíti a kiadást és 200-as válaszkóddal visszaadja a frissített adatokat.

```
public function destroy(Request $request)
{
    $kiadas = Kiadas::find($request->kiadasID);
    if (is_null($kiadas)) {
        return response()->json(['message' => 'Nem található a kiadás'], 400);
    }
    $kiadas->delete();
    return response()->json(['message' => 'Törölve lett a rekord.'], 204);
}
```

64. kép (Kontrollerek - Backend)

destroy(Kiadas \$kiadas): Törli a megadott kiadást (Kiadas rekord). Ha a kiadás nem található, 400-as hibát ad vissza a "Nem található a kiadás" üzenettel. Ha sikeres a törlés, 204-es válaszkóddal válaszol (nincs tartalom, de a törlés megtörtént).

KiadasKategoriakController

A KiadasKategoria kontroller, a Kiadas_kategoria modellel kapcsolatos műveleteket végez. A metódusok az alábbiakat végzik el:

```
public function index()
{
    return Kiadas_kategoria::all();
}
```

65. kép (Kontrollerek - Backend)

index(): Az összes Kiadas_kategoria rekordot lekérdezi a rendszerből és JSON formátumban visszaadja.

```
public function show(Request $request)
{
    $kiadas = Kiadas_kategoria::find($request->id);
    if(is_null($kiadas)){
        return response()->json(['message' => 'Felhasznalo nem talalhato'],404);
    }
    return response()->json($kiadas,200);
}
```

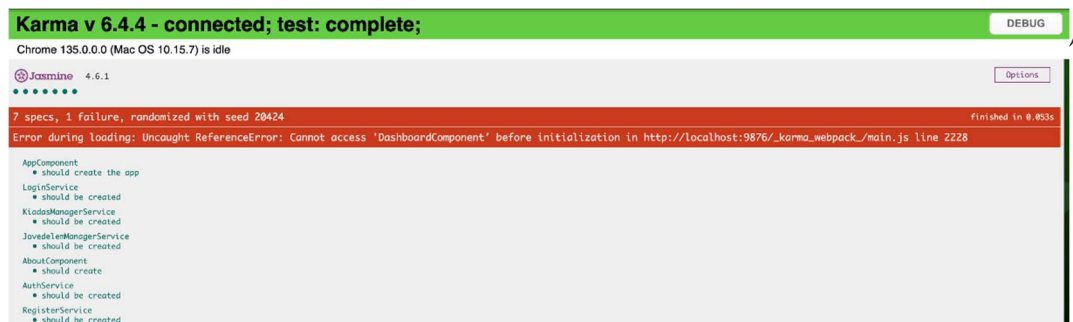
66. kép (Kontrollerek - Backend)

show(Request \$request): Egy adott **Kiadas_kategoria** rekordot kér le az id paraméter alapján, amelyet a kérés tartalmaz. Ha nem található a keresett kiadás kategória, akkor 404-es hibát ad

vissza a "Felhasználó nem található" üzenettel. Ezt a metódust akkor használjuk, ha egy konkrét kategóriát szeretnénk lekérdezni a rendszerből.

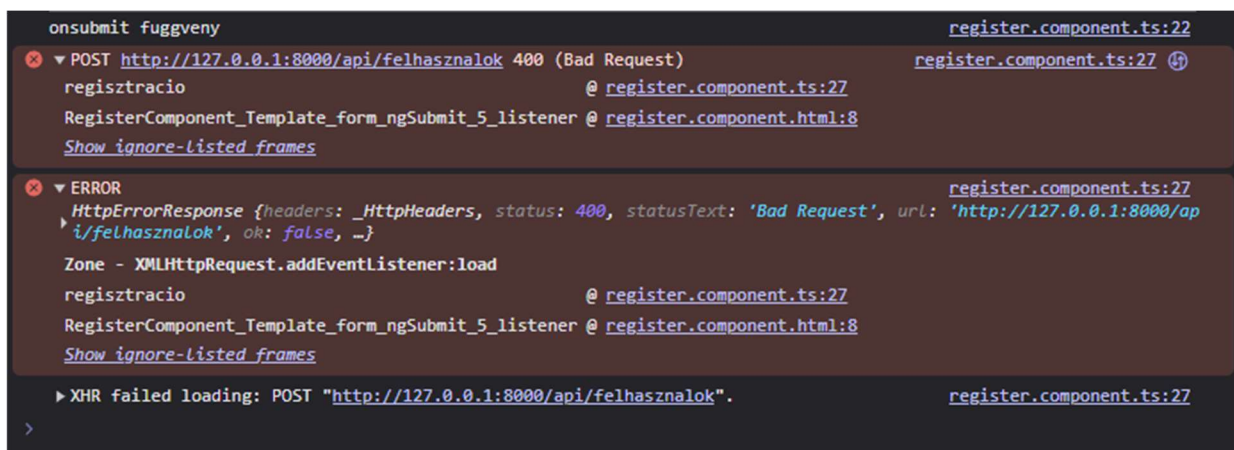
Tesztelés

Frontend tesztelés



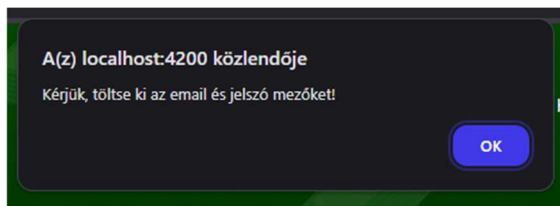
Egyetlen egy hiba áll fent, a Dashboarddal kapcsolatosan, ami folyamatosan fennálló hiba. A guard figyel, hogy a felhasználó be van-e jelentkezve, ha igen akkor engedélyt kap arra, hogy beléphessen az irányítópultba. Ezt a teszt nem tudja futtatni, figyelni, ezért nem is nézi.

Üres regisztráció során, a következő hiba jön fel, mivel nem adtunk meg semmilyen adatot, amit felvihetnénk a rendszerbe.

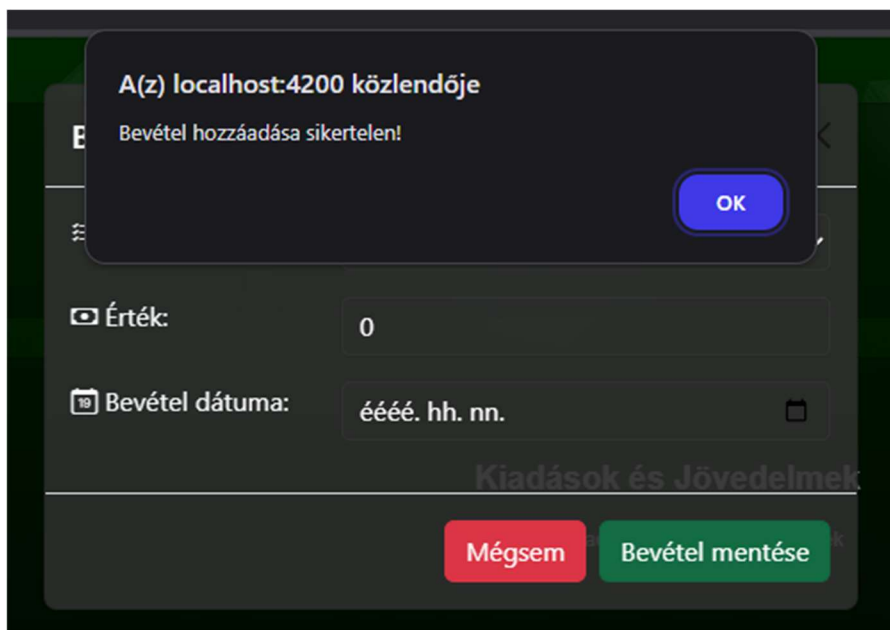


68. kép (Tesztelés)

Üresen hagyott bejelentkezés során, egy felugró ablak jelenik meg az oldalon.



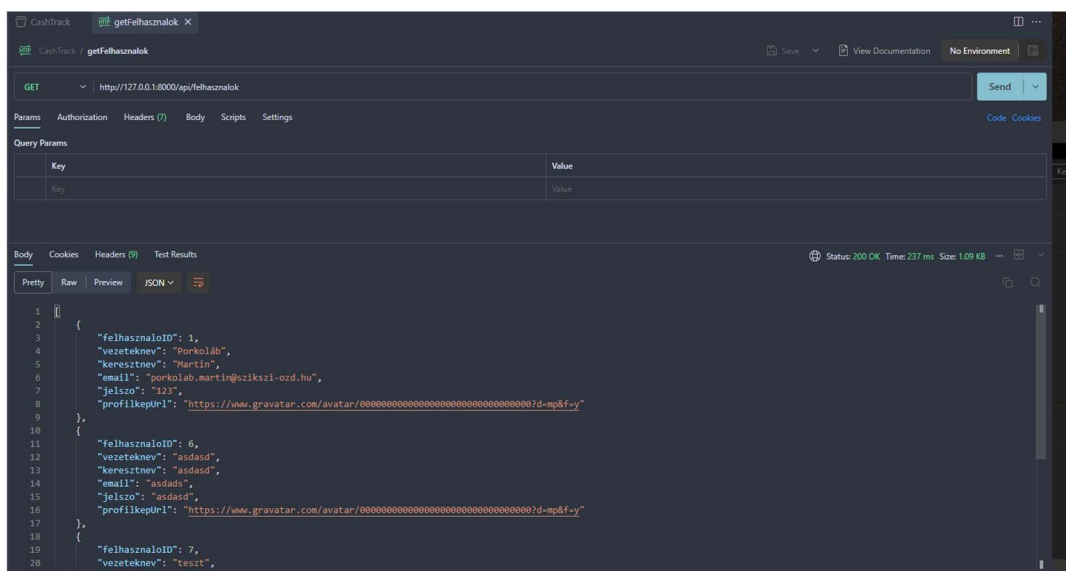
Üresen hagyott mezők a bevétel és kiadás hozzáadásánál, egy felugró ablakot okoz, hibaüzenettel.



70. kép (Tesztelés)

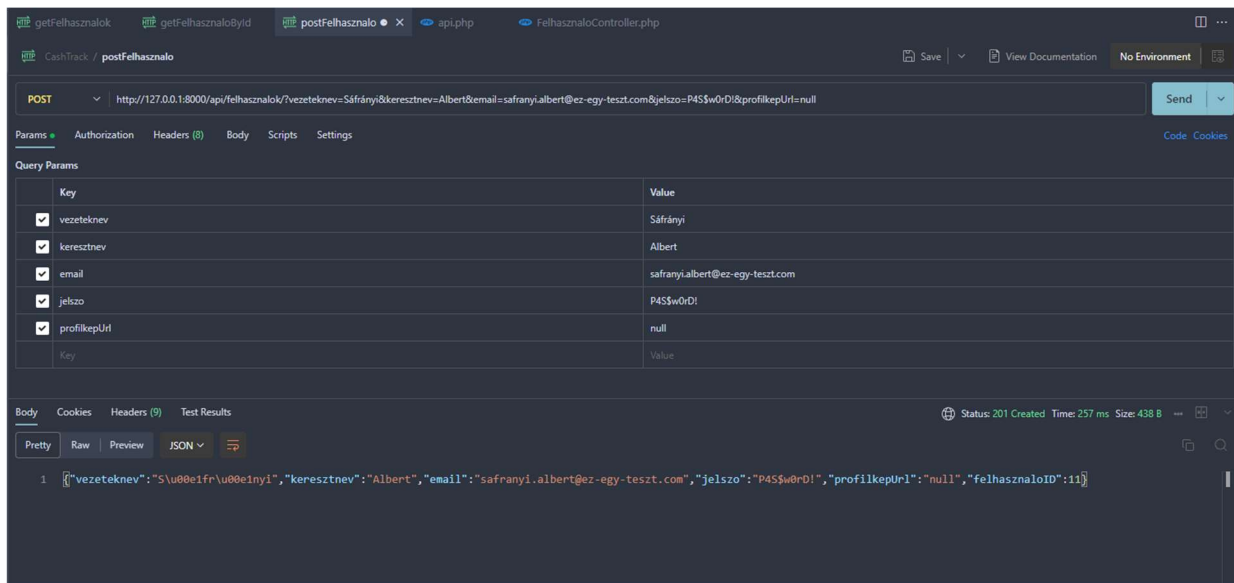
Backend tesztelés

A backendet Thunder Clienttel, valamint Postman API-val is leteszteltük. A következő képeken az eredmények találhatók.

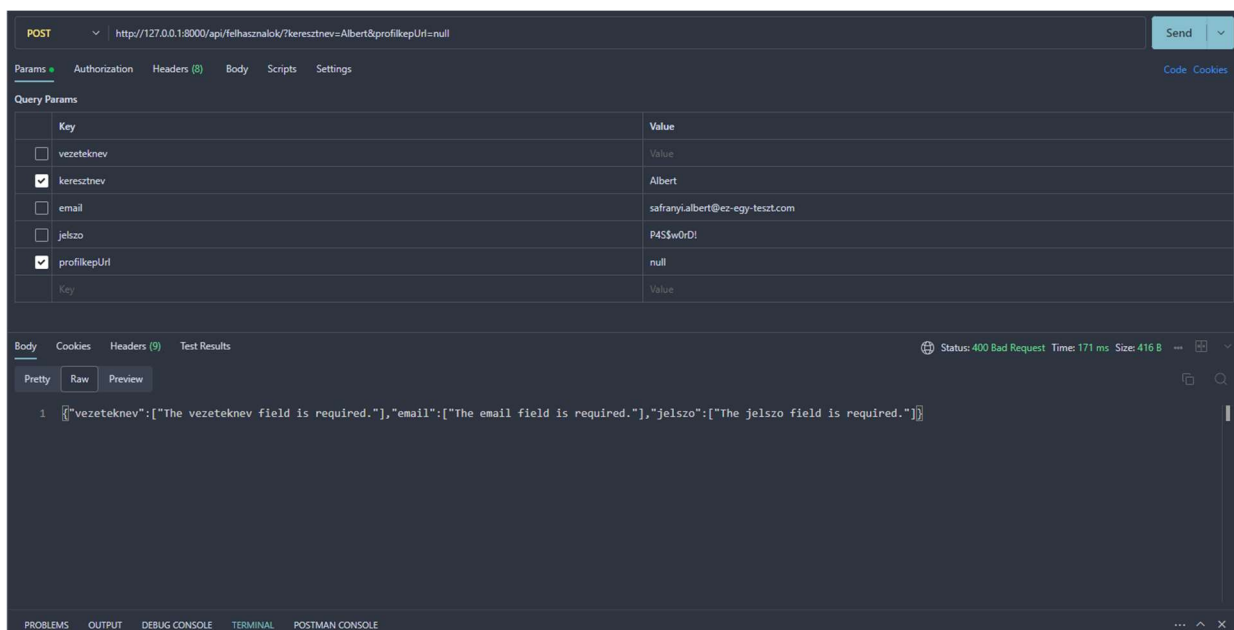


71. kép (Tesztelés)

A 71. képen található teszt lekéri az összes felhasználót az adatbázisból.

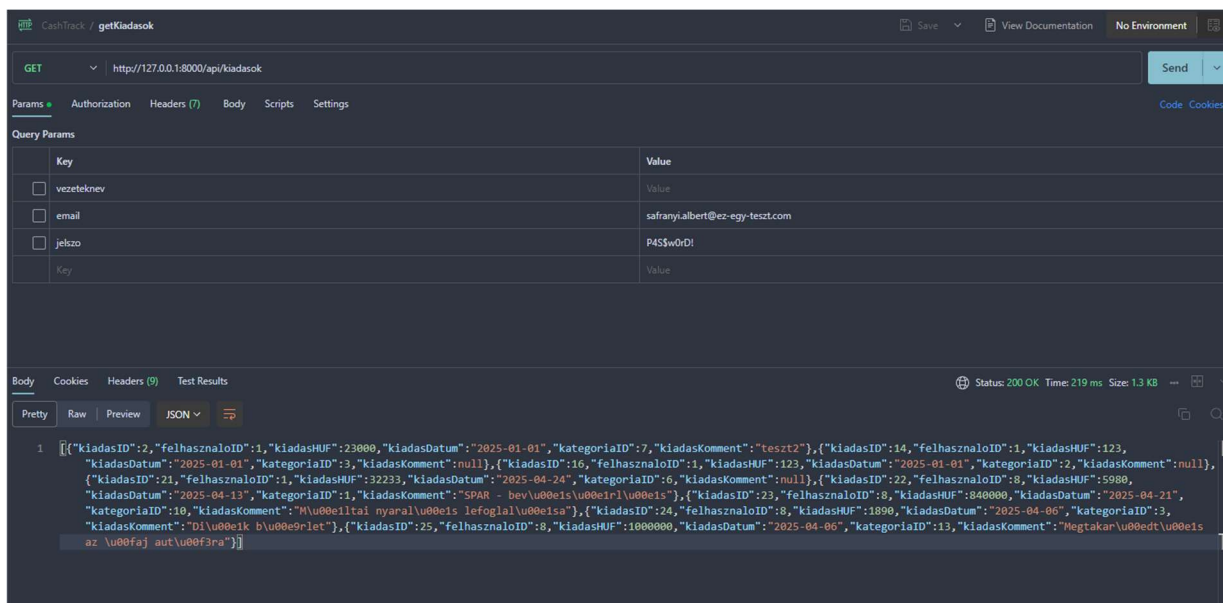


72. kép (Tesztelés)



73. kép (Tesztelés)

A 72. és 73. képen a felhasználó feltöltéséről való tesztet láthatjuk.



74. kép (Tesztelés)

A 74. képen az összes kiadás lekérése történik az adatbázisból.

Még rengeteg tesztel rendelkezőnk, mindenféle teszteléssel, ezek a legfontosabb tesztjeink.

Továbbfejlesztési lehetőségek

Funkcionalitás bővítése

- Felhő alapú szinkronizáció
- Jelenleg az adatok helyileg tárolódnak. Egy Firebase vagy Supabase alapú megoldás lehetővé tenné a több eszköz közötti adatmegosztást.
- Kiadási statisztikák és vizualizációk
- Exportálási lehetőségek bővítése Jelenleg JSON formátumban menti az adatokat, de érdemes lenne CSV vagy PDF exportálást is hozzáadni.
- AI-alapú költségi elemzés
- Egy mesterséges intelligencia modul felismerhetné a kiadási szokásokat és tippeket adhatna a megtakarításhoz.

Felhasználói élmény javítása

- A felület finomhangolása mobilon és tableten való kényelmesebb használathoz.
- Dark mode támogatás - Sötét mód bevezetése a jobb vizuális élmény érdekében.

- PWA fejlesztések
- Teljes offline működés támogatása és gyorsítótárzás fejlesztése.

Biztonság és adatvédelem

- Felhasználói hitelesítés és fiókok.
- Jelenleg nincs bejelentkezési rendszer, de egy Google/Facebook OAuth vagy saját hitelesítés biztonságosabbá tenné az alkalmazást.
- Titkosítás és adatvédelem.
- Az érzékeny pénzügyi adatok titkosított tárolása (pl. AES vagy bcrypt használata).
- Biztonsági mentések.
- Automatikus biztonsági mentés funkció Google Drive-ba vagy más felhőszolgáltatásba.

Backend és technológiai fejlesztések

- Jobb állapotkezelés.
- Ha az alkalmazás bővül, egy NgRx vagy Redux állapotkezelő bevezetése hatékonyabbá tenné a működését.
- Tesztautomatizálás.
- Unit tesztek (Jasmine/Karma) és E2E tesztek (Cypress) hozzáadása a stabilitás növelésére.

Irodalomjegyzék

- <https://www.npmjs.com/package/ng2-charts>
- <https://getbootstrap.com>
- <https://angular.dev>
- <https://laravel.com>
- <https://www.w3schools.com>
- <https://spline.design/>
- <https://formsubmit.co/>

- <https://michalsnik.github.io/aos/>